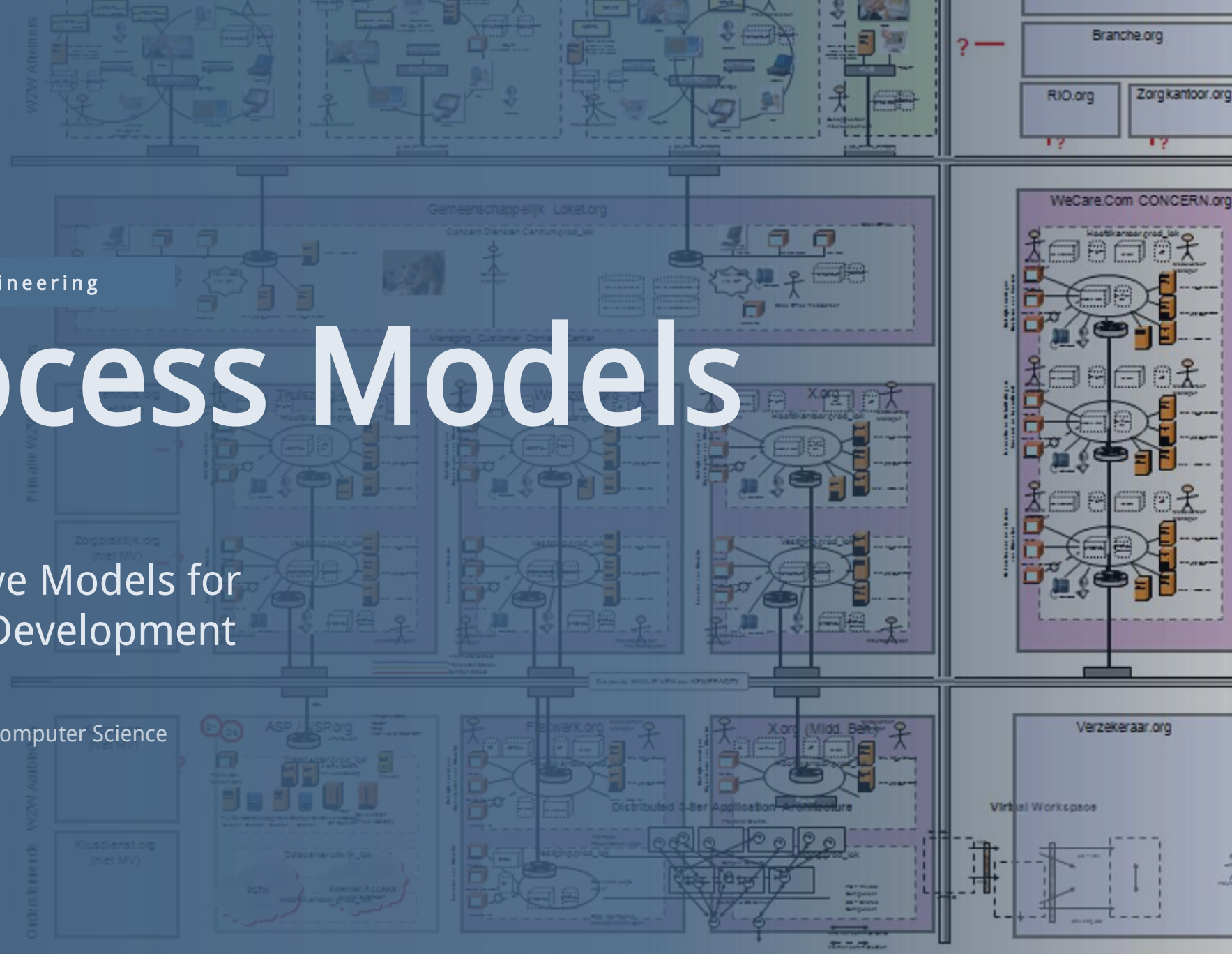


Software Engineering

Prescriptive Models for Software Development

 2025 Computer Science

WZV-Jveta: reges

Store WZY paidbede

WZY-Flickers

[illegible][Veronica's Data Communication Principles](#)

1. `GLSL`, `OpenGL` en `OpenGL` - een kleine
multimediale en kleine taal.
2. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.
3. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.
4. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.
5. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.
6. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.
7. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.
8. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.
9. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.
10. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.
11. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.
12. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.
13. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.
14. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.
15. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.
16. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.
17. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.
18. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.
19. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.
20. `GLSL` staat voor `OpenGL Shading Language` en `OpenGL`.

Software & Hardware - principles

```

1)Kopplungen fuer Applikationen werden mittels
  derXSL2SLAP in derXSL2SLAP, in derXSL2SLAP
2)Applikationen werden mittels derXSL2SLAP in derXSL2SLAP
  platform in derXSL2SLAP
3)Applikationen werden mittels derXSL2SLAP in derXSL2SLAP
  in derXSL2SLAP
4)Applikationen werden mittels derXSL2SLAP in derXSL2SLAP
  in derXSL2SLAP
5)Applikationen werden mittels derXSL2SLAP in derXSL2SLAP
  in derXSL2SLAP
6)Applikationen werden mittels derXSL2SLAP in derXSL2SLAP
  in derXSL2SLAP
7)Applikationen werden mittels derXSL2SLAP in derXSL2SLAP
  in derXSL2SLAP
8)Applikationen werden mittels derXSL2SLAP in derXSL2SLAP
  in derXSL2SLAP
9)Applikationen werden mittels derXSL2SLAP in derXSL2SLAP
  in derXSL2SLAP
10)Applikationen werden mittels derXSL2SLAP in derXSL2SLAP
  in derXSL2SLAP

```

Contributors

● *multisystemic, multi-national* - 10 years

ICT-Services 1 (online)

Overall Technische Architektur-entwürfe

Contributors

Presentation Overview

6 Chapters

01 Prescriptive Process Models

Overview of software development methodologies

03 Incremental Models

RAD and incremental approaches

05 Specialized Models

Component-based and formal methods

02 Waterfall Model

Classic linear sequential lifecycle

04 Evolutionary Models

Prototyping and spiral models

06 Unified Process

UP framework and lifecycle

Prescriptive Process Models

Waterfall Model

Classic Life Cycle: Linear sequential approach with distinct phases from communication through deployment.

- Fixed requirements
- Linear progression
- Document-driven

Specialized Models

Component-Based & Formal: Emphasize reuse and mathematical specification.

- Component reuse
- Formal methods
- Aspect-oriented

Incremental Models

RAD & Incremental: Deliver software in increments with parallel development teams for rapid delivery.

- Modular development
- Parallel teams
- Fast delivery

Unified Process (UP)

Use-Case Driven, Architecture-Centric: Iterative and incremental framework aligned with UML.

- ✓ **Four Phases:** Inception, Elaboration, Construction, Transition
- ✓ **Five Workflows:** Requirements → Analysis → Design → Implementation → Test

Evolutionary Models

Prototyping & Spiral: Accommodate change through iterative cycles and risk-driven development.

- Iterative nature
- Risk reduction
- Change accommodation



Key Insight

All process models perform the same set of generic framework activities—**Communication, Planning, Modeling, Construction, and Deployment**—but suggest different process flows and emphasize different aspects of software development.

Waterfall Model

Classic Life Cycle

01 Communication

Establish project vision and gather stakeholder requirements

02 Planning

Define project roadmap and resource allocation strategy

03 Modeling

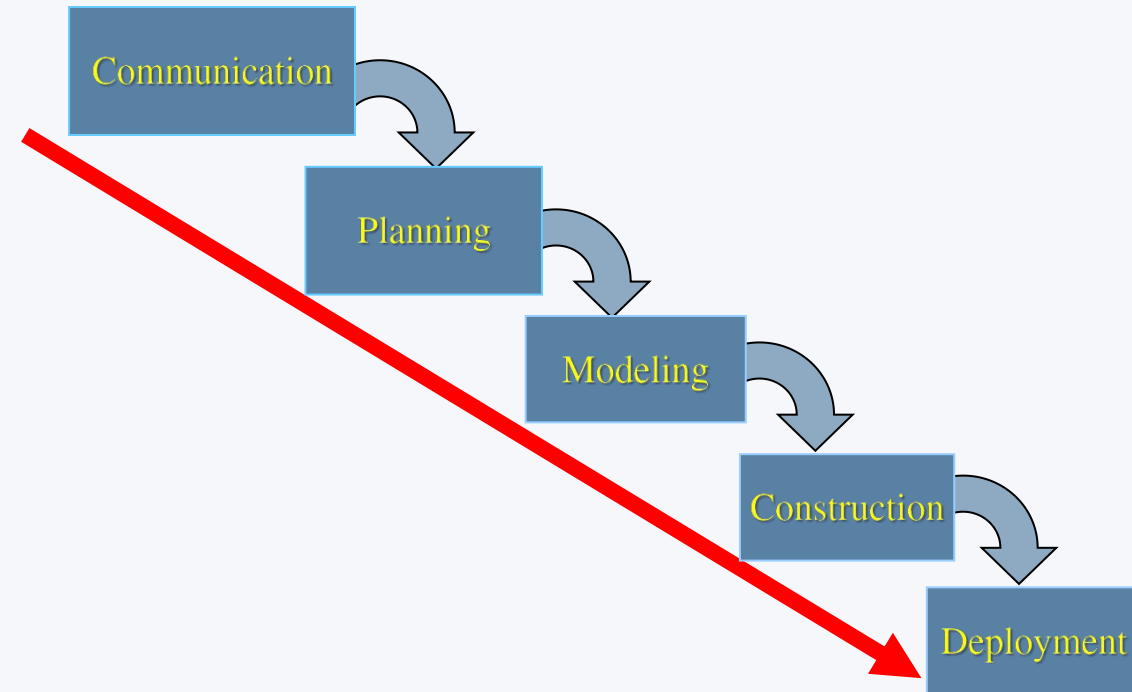
Create abstract representations of the system

04 Construction

Transform design into executable software

05 Deployment

Deliver system to operational environment



→ Linear Sequential Flow

Each phase must be completed before proceeding to the next.
No overlap or iteration between phases. Changes are difficult to accommodate once a phase is completed.

Waterfall Model

Strengths and Limitations

✓ Merits

Predictable Structure

Serves as a useful process model in situations where **requirements are fixed and work proceeds to completion in a linear manner**.

Provides clear milestones and deliverables for each phase.

Well-Documented

Each phase produces comprehensive documentation that provides clear understanding of progress and deliverables. **Easy to understand and manage** due to rigid structure.

Clear Requirements

Forces stakeholders to explicitly state all requirements upfront, reducing ambiguity. Works well for **projects with stable and well-understood requirements**.

⚠ Demerits

Inflexible to Change

Real projects rarely follow the sequential flow. Changes can cause confusion as the project team proceeds, making it difficult to accommodate evolving requirements.

Late Feedback

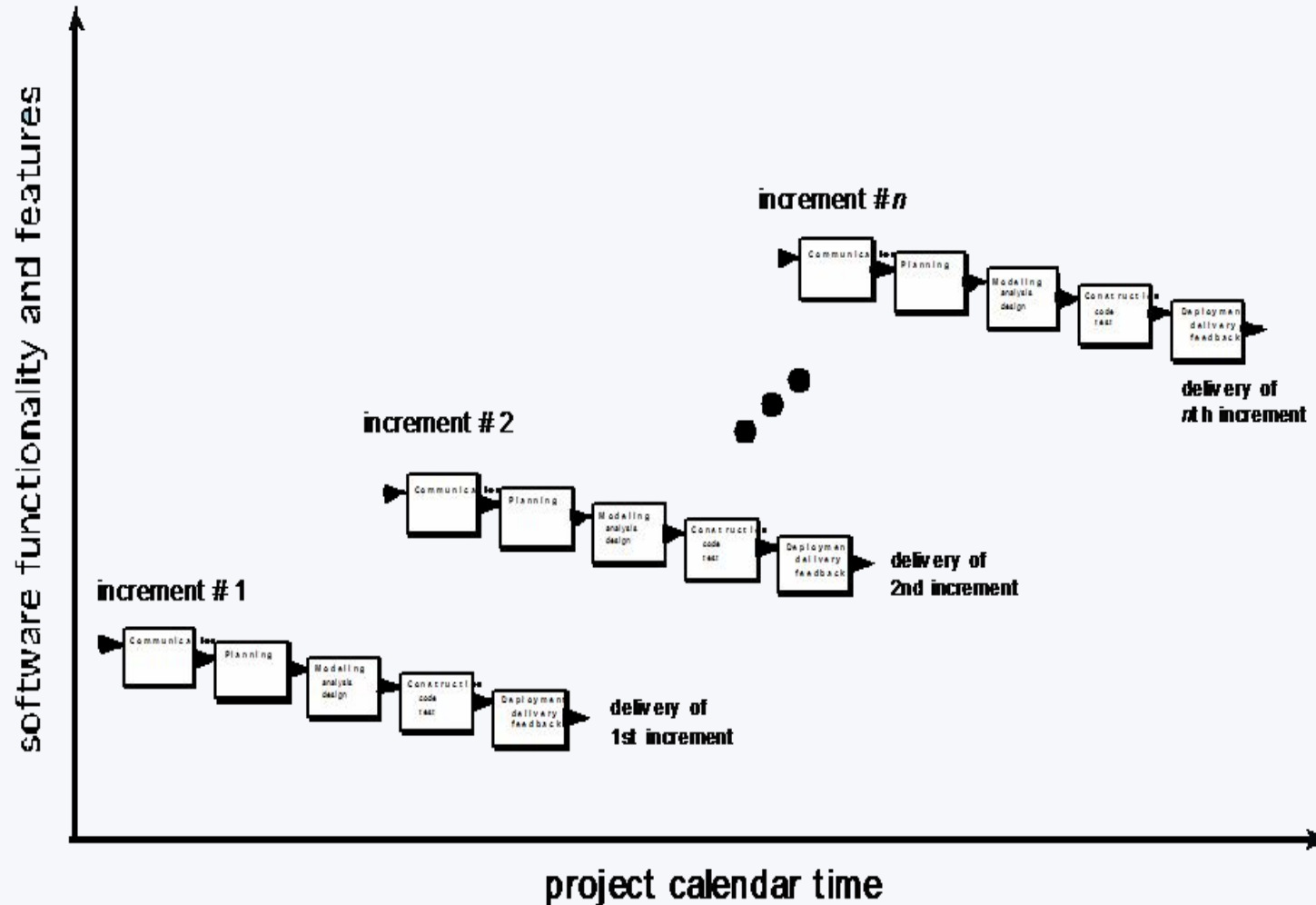
A working version of the program will not be available until late in the project time-span. This delays feedback and increases risk of building the wrong system.

High Uncertainty

The model requires customer to state all requirements explicitly and has difficulty accommodating the natural uncertainty that exists at the beginning of many projects.

Incremental Model

Building Systems in Stages



Core Principle

Deliver software as a series of incremental releases, where each increment provides additional functionality. Early increments contain core architecture and essential features.

Key Characteristics

- ✓ Each increment delivers operational product
- ✓ Core functionality delivered early
- ✓ Parallel development possible
- ✓ Risk reduction through early delivery

Development Flow

Increment 1: Core architecture + basic features
Increment 2: Add secondary functionality
Increment 3: Additional features and refinement
Increment N: Complete system with all features

Incremental Model

Strengths and Limitations

✓ Merits

Resource Management

Useful when **complete implementation staffing is unavailable by business deadline**. Allows teams to work on different increments simultaneously.

Risk Mitigation

Useful for planning to **manage technical risks** through incremental delivery. Early increments identify architectural and integration issues sooner.

Early Value

Focus on delivery of an operational product with each increment, allowing customers to realize value early in the development process.

⚠ Demerits

Neglect Risk

Early increments would be **"ignored"** easily if focus is solely on delivery. Teams may neglect architectural integrity of early increments.

Integration Complexity

Requires careful planning to ensure increments integrate properly. **Later increments may require refactoring of earlier ones**, increasing effort.

Scope Creep

Each increment may encourage additional feature requests, potentially leading to scope creep and extended timelines.

2-6

Typical Increments

Most projects use 3-4 increments

3-12

Months per Increment

Duration varies by complexity

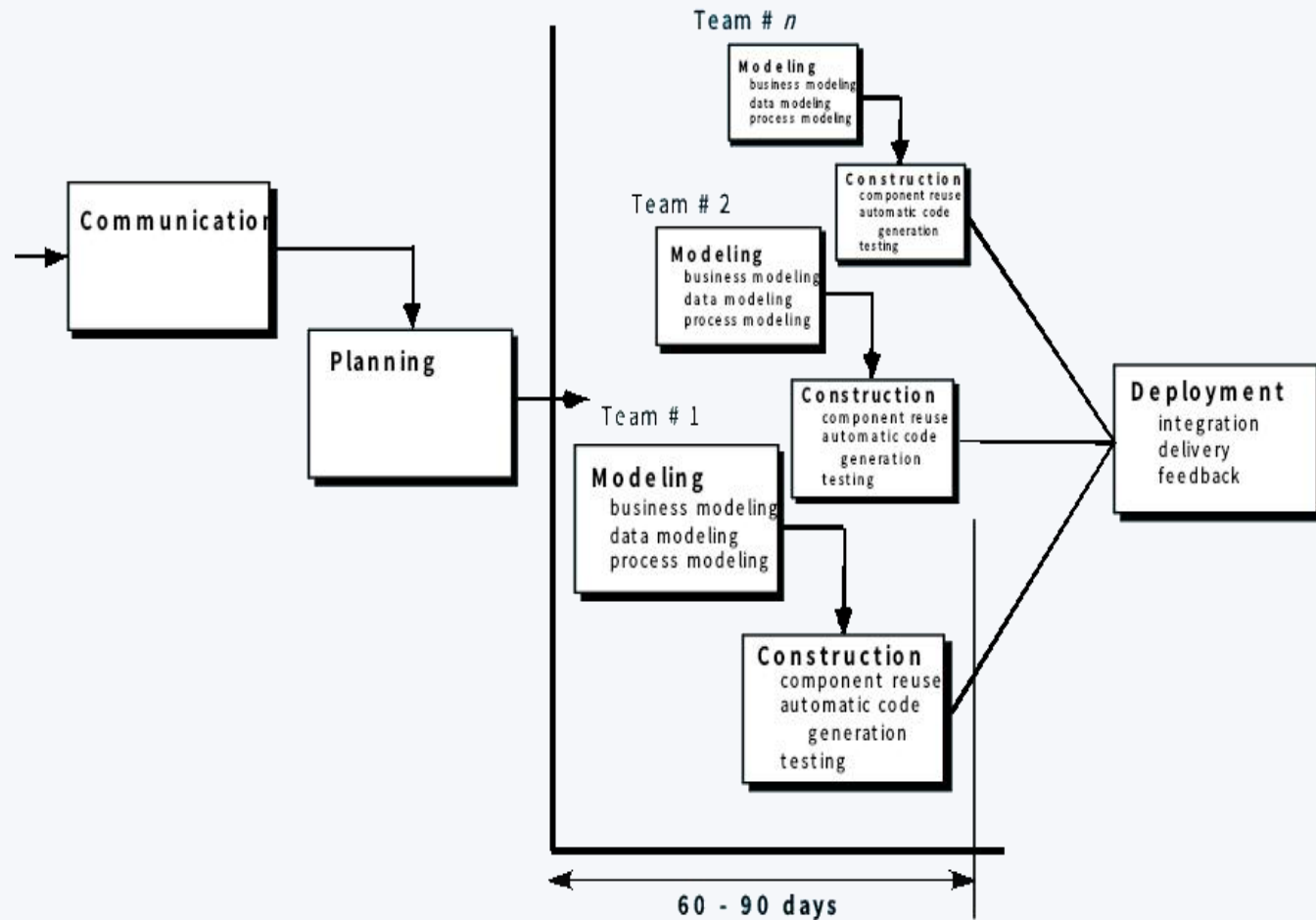
30-50%

Early Value Delivery

Core functionality available first

RAD Model

Rapid Application Development



🚀 High-Speed Approach

RAD is a high-speed adaptation of the linear sequential model in which rapid development is achieved by using component-based construction and parallel development teams.

Core Principles

- 👥 **Parallel Teams:** Multiple teams work simultaneously
- 🔧 **Modular Components:** System built from reusable parts
- 🕒 **Time-boxed:** Tight deadlines (60-90 days)
- ⚙️ **Automated Tools:** CASE tools and visual development



Typical Timeline
60-90 Days

RAD Model

Strengths and Limitations

✓ Merits

Rapid Delivery

Enables a development team to create a **"fully functional system" within a very short time period**. Ideal for projects with tight deadlines and clear scope.

Parallel Development

Multiple teams working simultaneously on different modules significantly accelerates development. **Component reusability** reduces development time.

Customer Engagement

Continuous customer involvement through iterative reviews and feedback ensures the system meets requirements. **Reduces risk of building wrong system**.

60-90

Days Timeline

Typical RAD duration

2-8

Parallel Teams

Working simultaneously

⚠ Demerits

Resource Intensive

For large, but scalable projects, RAD **requires sufficient human resource** to create the right number of RAD teams. Requires multiple skilled developers working in parallel.

Modularization Dependency

Building components necessary for RAD will be problematic if a system cannot be properly modularized. Tightly coupled systems are not suitable for RAD.

Performance & Risk Limitations

The RAD approach may not work when **high performance is an issue**. RAD may not be appropriate when **technical risks are high**.

70-80%

Reuse Rate

Component utilization

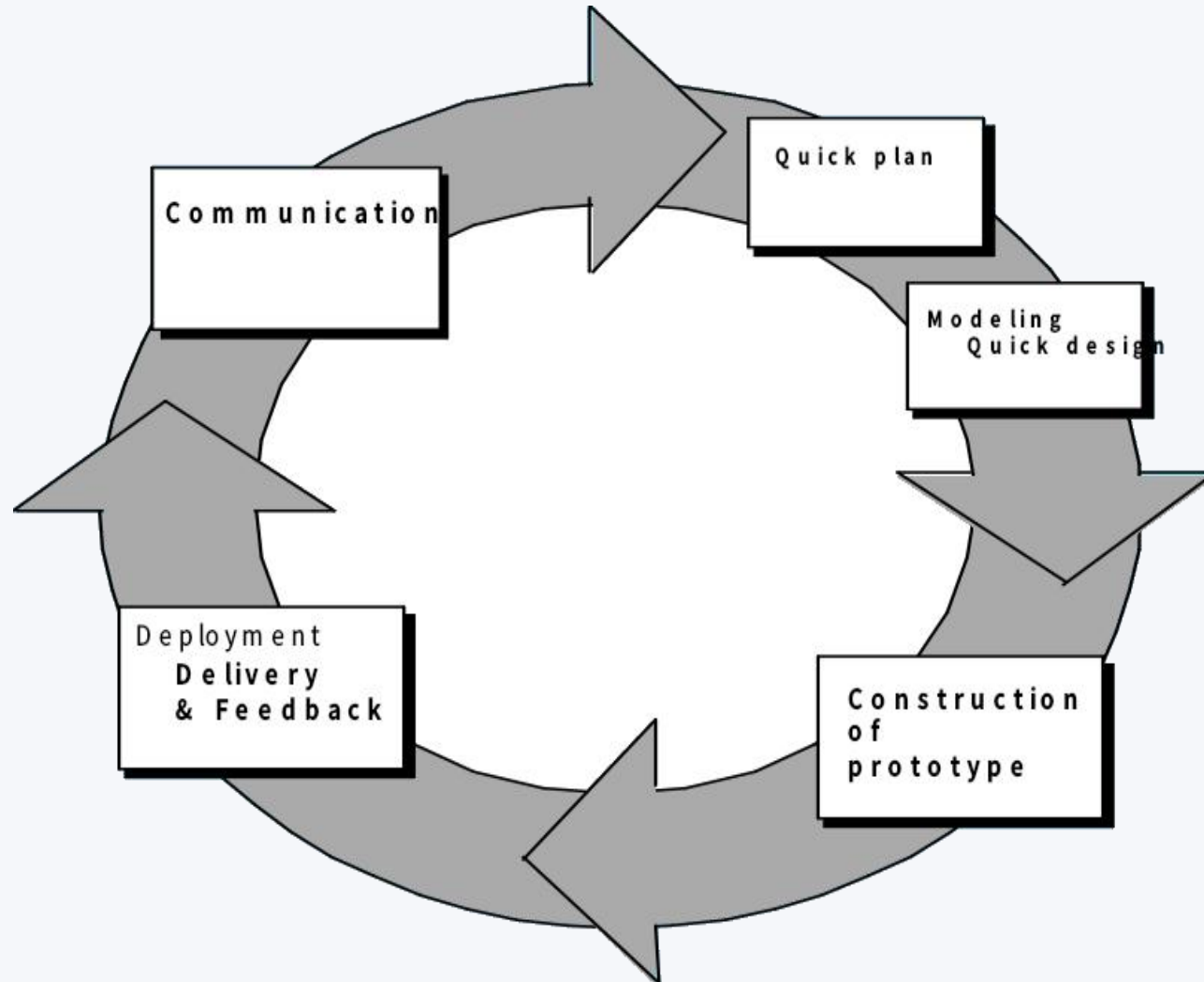
5-10×

Speed Gain

vs traditional methods

Evolutionary Models

Prototyping: Iterative Exploration



🔄 Iterative Refinement

The prototyping model creates **partial implementations** to help developers and customers better understand requirements when they are fuzzy or unclear.

Prototyping Cycle

- 1 **Identify Requirements:** Gather initial, often incomplete, requirements
- 2 **Develop Prototype:** Build quick, partial implementation
- 3 **Customer Evaluation:** Users test and provide feedback
- 4 **Refine Requirements:** Incorporate feedback and clarify needs
- 5 **Iterate or Discard:** Continue cycle or move to production

When to Use

Ideal when requirements are **fuzzy, unclear, or evolving**. Helps resolve uncertainty and explore technical feasibility before full commitment.

Prototyping Model

Strengths and Limitations

✓ Merits

Requirements Clarification

Serves as a mechanism for assisting developers and customers to **better understand what is to be built** when requirements are fuzzy.
Reduces ambiguity through tangible examples.

Risk Reduction

Prototypes help **explore technical feasibility** and validate critical assumptions early. Identifies potential problems before significant investment in development.

User Engagement

Provides tangible system for users to interact with, generating **more meaningful feedback** than abstract requirements discussions.
Improves user satisfaction.

⚠ Demerits

Scope Creep Risk

Customers always ask developers to make the prototype a **working product**, which makes software development management difficult.
Hard to distinguish prototype from final system.

Technical Compromises

Developers often make implementation compromises in order to get a prototype working quickly. These less-than-ideal choices may become integral parts of the system.

Additional Cost

Building prototypes requires additional time and effort. If not managed properly, prototyping can increase overall project cost and extend timelines.

2-6

Prototype Iterations
Typical refinement cycles

20-40%

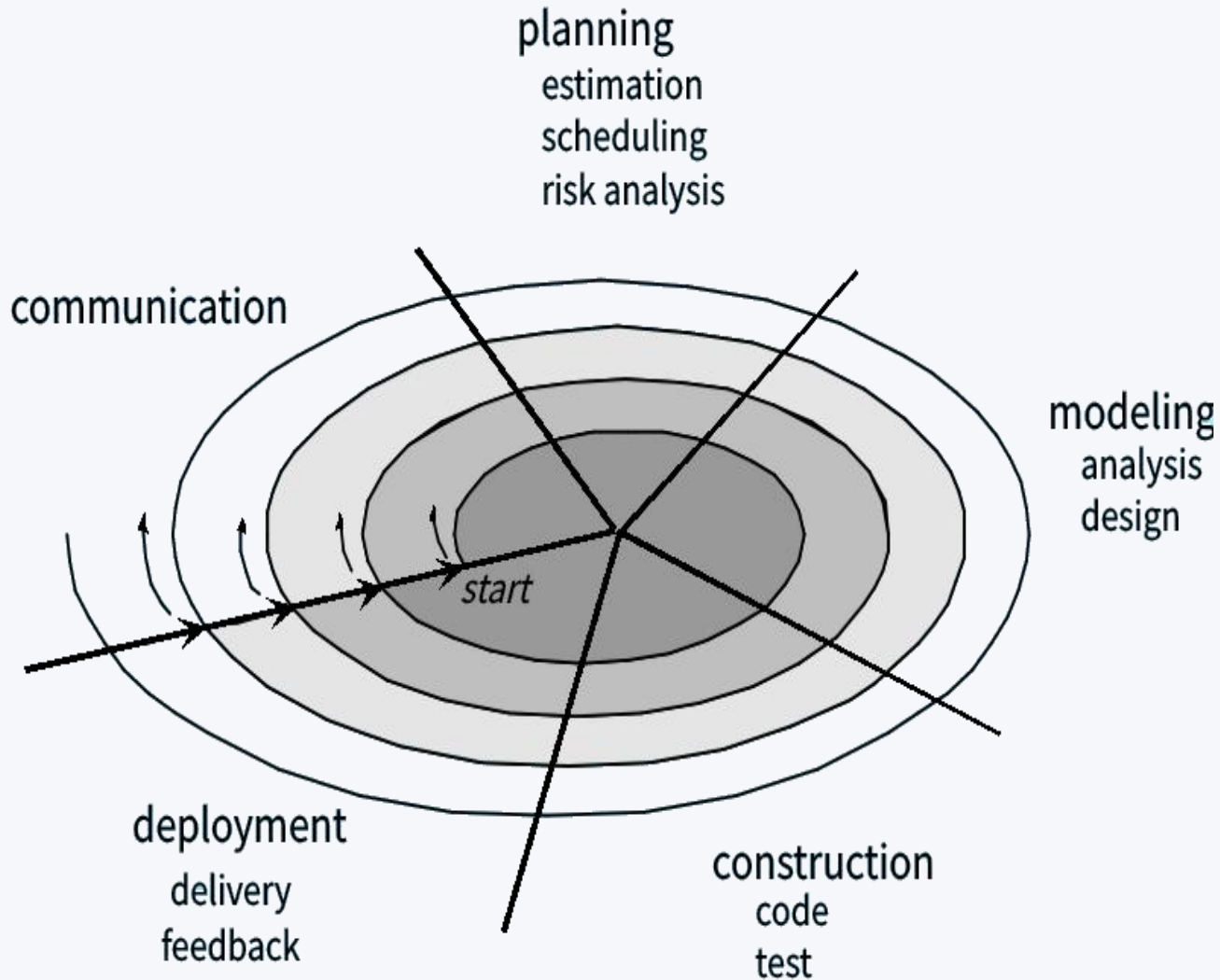
Requirements Clarity
Improvement in understanding

50-70%

Risk Reduction
Early issue identification

Spiral Model

Risk-Driven Evolutionary Development



Risk-Driven Approach

The spiral model combines **iterative development** with **systematic risk management**. Each cycle involves planning, risk analysis, engineering, and customer evaluation.

Four Quadrants

- 1 Planning:** Define objectives and constraints
- 2 Risk Analysis:** Identify and mitigate risks
- 3 Engineering:** Develop and test solution
- 4 Evaluation:** Customer reviews and feedback

Key Features

- Risk-driven cycles
- Prototyping support
- Iterative refinement
- Customer involvement

Spiral Model

Strengths and Limitations

✓ Merits

Proactive Risk Management

Uses prototyping as a risk reduction mechanism, enabling the developer to apply the prototyping approach at any stage. Risks are identified and mitigated early.

Change Accommodation

Enables the developer to apply the evolutionary approach at any stage in the evolution of the product. Changes can be incorporated in subsequent cycles.

Continuous Customer Involvement

Regular customer evaluation ensures the system meets evolving needs. Reduces risk of building the wrong system through frequent feedback.

⚠ Demerits

Perceived Lack of Control

It may be difficult to convince customers that the evolutionary approach is controllable. The iterative nature can appear unstructured and unpredictable.

Risk Management Dependency

If a major risk is not uncovered and managed, problems will undoubtedly occur. Success heavily depends on effective risk assessment and mitigation.

Complexity and Cost

Requires skilled risk analysts and may be more expensive than other models. Not suitable for small or low-risk projects due to overhead.

3-6

Typical Cycles

Most projects complete in 3-6 spirals

60-80%

Risk Reduction

Major risks identified early

High

Complexity

Requires skilled risk analysts

Specialized Process Models

Domain-Specific Approaches

Component-Based Development

Emphasizes component reuse and assembly. The process to apply when reuse is a development objective. Focuses on integrating existing, validated components rather than building from scratch.

- **Library-driven:** Extensive use of component repositories
- **Integration-focused:** Emphasis on assembly over coding
- **Quality-assured:** Leverages tested, proven components

Aspect-Oriented Software Development (AOSD)

Provides a process and methodological approach for defining, specifying, designing, and constructing aspects. Separates cross-cutting concerns (logging, security, transactions) from business logic.

- **Cross-cutting concerns:** Manages tangled functionality
- **Modularization:** Clean separation of aspects
- **Weaving:** Automatic composition of aspects

Formal Methods Model

Emphasizes the mathematical specification of requirements. Uses formal mathematical notation and verification to prove system correctness. Critical for safety-critical and mission-critical systems.

- **Mathematical rigor:** Formal proofs of correctness
- **Error-free:** Eliminates ambiguity and inconsistency
- **Critical systems:** Used in aerospace, medical, nuclear

Unified Process (UP)

A "use-case driven, architecture-centric, iterative and incremental" software process closely aligned with the Unified Modeling Language (UML). Comprehensive framework for object-oriented development.

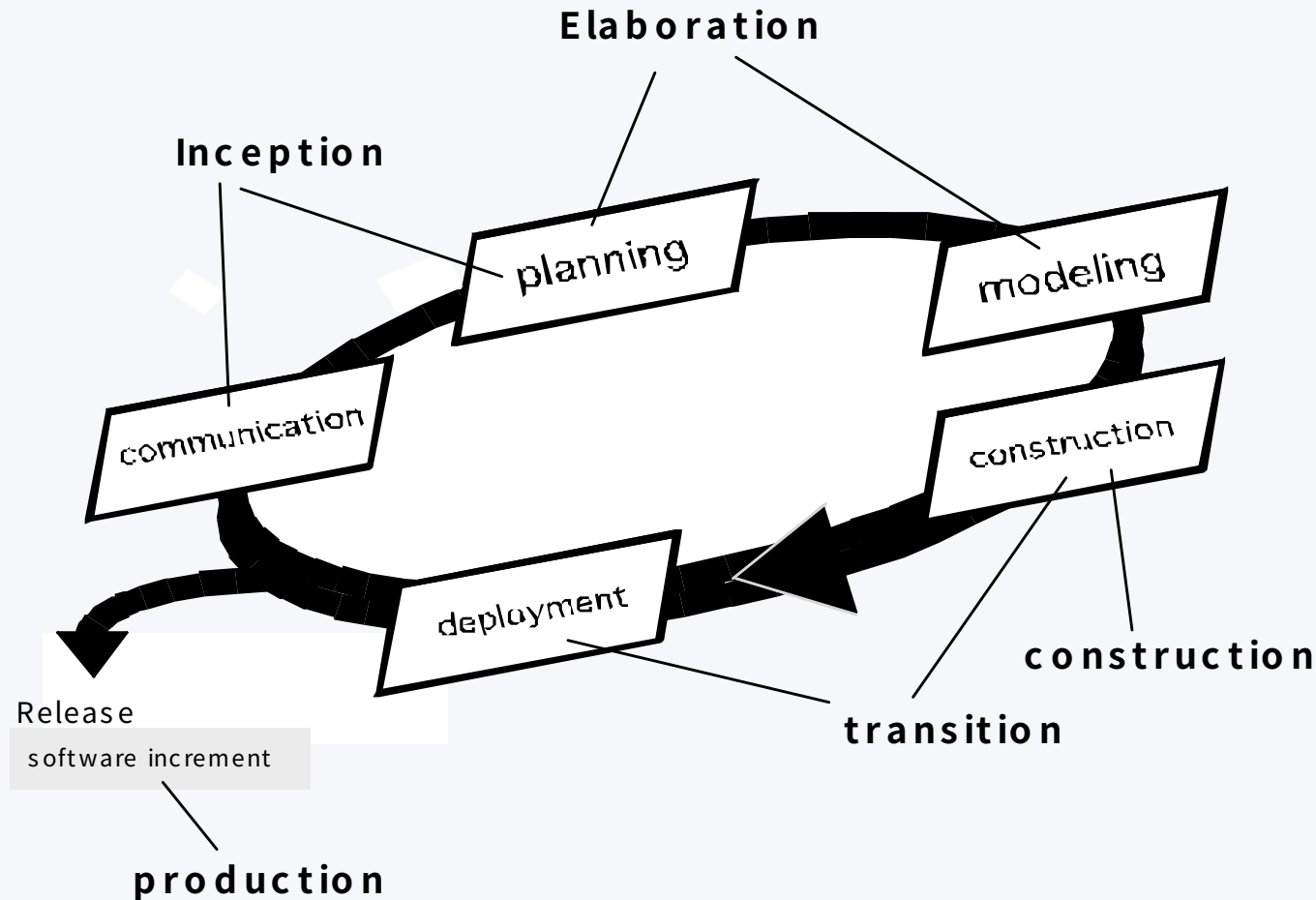
- **Use-case driven:** Requirements captured as use cases
- **Architecture-centric:** Focus on system structure
- **UML-aligned:** Integrated with modeling language

Selection Criteria

Choose the process model based on project characteristics: **Component-Based** for reuse-focused projects, **Formal Methods** for safety-critical systems, **AOSD** for systems with cross-cutting concerns, and **Unified Process** for comprehensive OO development.

Unified Process (UP)

Use-Case Driven, Architecture-Centric Framework



Iterative & Incremental

UP is a "use-case driven, architecture-centric, iterative and incremental" software process closely aligned with UML.

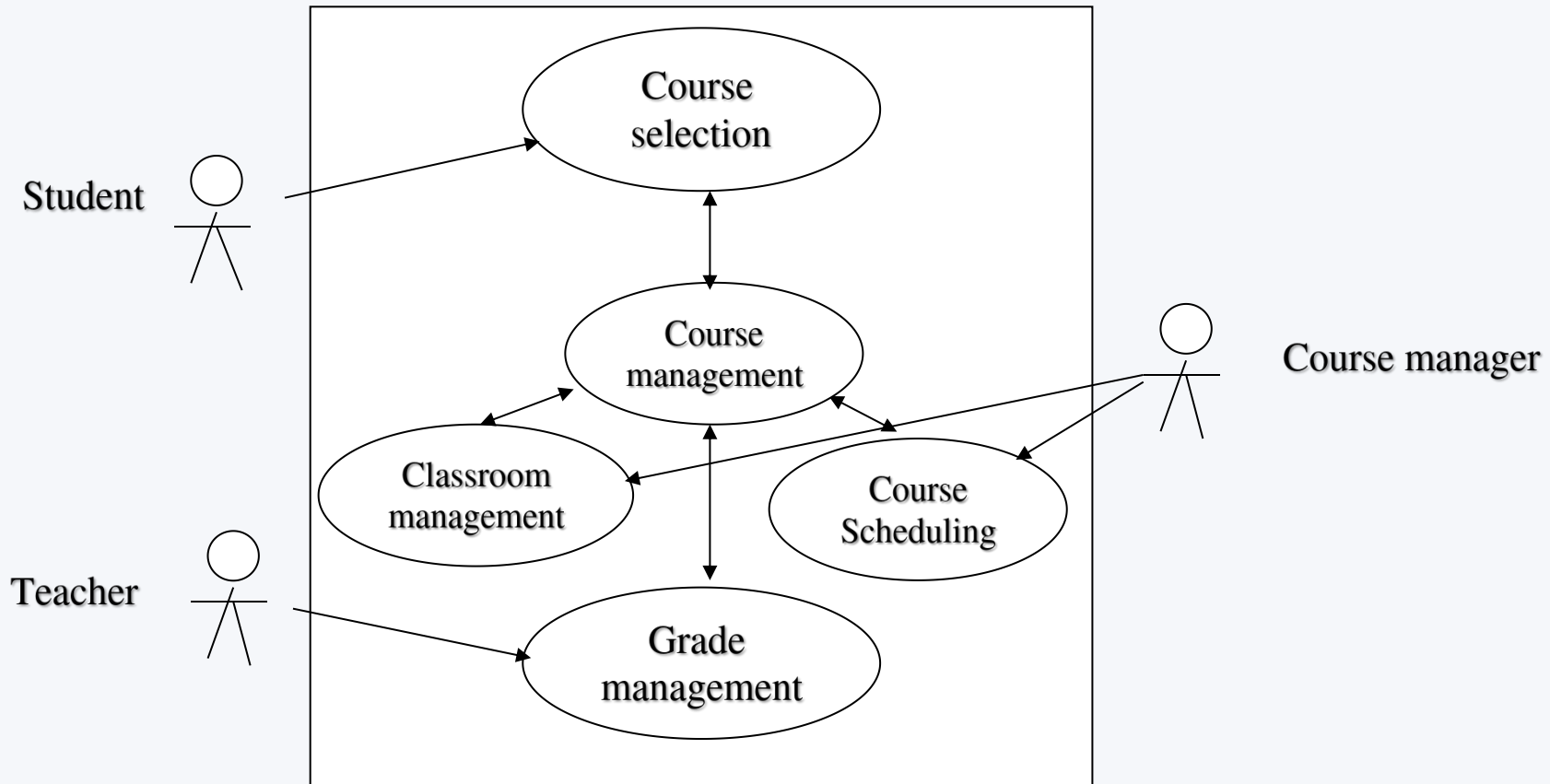
Four Phases

- 💡 **Inception:** Define project scope and vision
- 🔧 **Elaboration:** Plan and design architecture
- 🔨 **Construction:** Build the product
- 🔑 **Transition:** Deploy to users

Key Characteristics

- Use-case driven
- Architecture-centric
- Iterative development
- Risk-focused

Use Cases and Workflows



Unified Process

Five Core Workflows

01 Requirements

Capture functional requirements through use cases. Identify actors, define system boundaries, and specify what the system should do from user perspective.

- Use Case Model • Requirements specification • Actor identification • System boundaries

03 Implementation

Translate design into executable code. Implement classes and components according to design specifications. Perform unit testing and integration of developed modules.

- Component implementation • Unit testing • Integration • Source code management

05 Deployment

Deliver system to operational environment. Package software, install at customer site, train users, and transition system into production use.

- Deployment Model • Software packaging • Installation • User training • Production transition

02 Analysis & Design

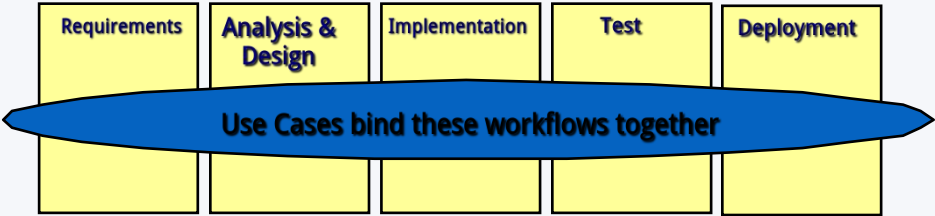
Create models that realize requirements. Transform use cases into analysis and design models. Define system architecture and detailed class specifications.

- Analysis Model • Design Model • Architecture definition • Class design

04 Test

Verify system meets requirements. Execute test cases derived from use cases. Perform integration testing, system testing, and acceptance testing.

- Test Model • Integration testing • System testing • User acceptance testing



Use Cases Bind Workflows Together

Each workflow implements the requirements captured in use cases. Use cases provide **traceability** from requirements through design, implementation, and testing activities.

Use Cases

Driving the Development Process

How Use Cases Drive Development

Capturing Functional Requirements

Use cases **capture the functional requirements** from user perspective. They describe what the system should do and how users interact with it.

Identifying Classes

Analysis of use cases helps in **identifying classes based on use cases**. Nouns become candidate classes; verbs become candidate methods.

Implementing Use Cases

Developers **implement use cases** by creating classes, methods, and interactions. Each use case realization guides the implementation.

Testing the Use Cases

Test cases are derived from use cases for **testing the use cases**. Acceptance criteria are defined based on use case success scenarios.

Planning Basis

Use cases provide **a basis for planning iterative development** by prioritizing features and estimating effort. They help define development cycles and milestones.

Documentation Foundation

Use cases provide **a foundation for user manuals** by describing how users interact with the system. They become the basis for help documentation and training materials.

Sales Units

Use cases can define **possible sales units** by packaging related functionality. They help identify system features that can be sold separately or as bundles.

Software Architecture

Definition and Scope

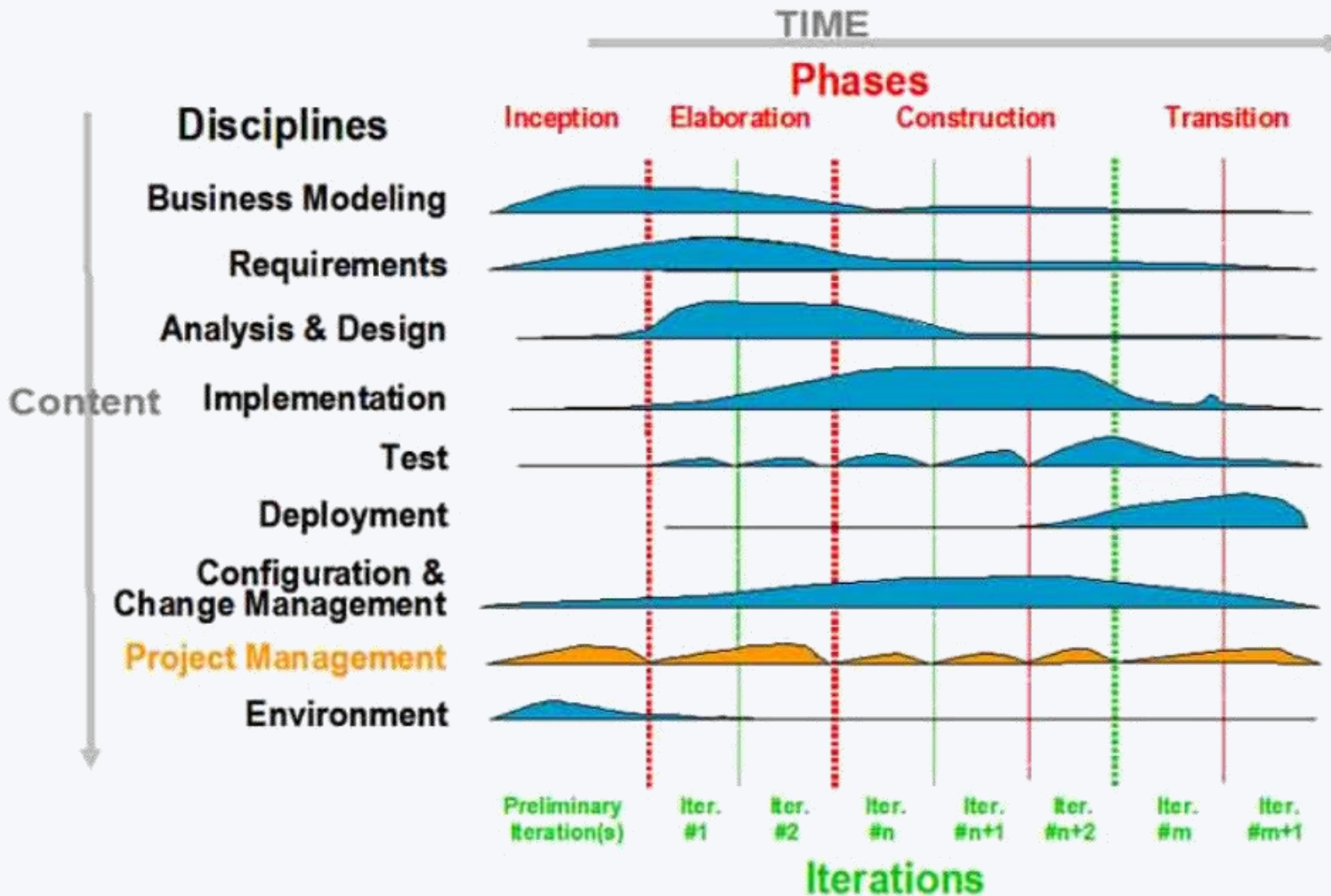
What is Software Architecture?

- System Organization**
Software architecture encompasses the significant decisions about **the organization of a software system**. It defines how components are structured and arranged.
- Structural Elements**
The **selection of the structural elements and their interfaces** by which the system is composed, together with their behavior as specified in the collaborations among those elements.
- Subsystem Composition**
The **composition of the structural and behavioral elements** into progressively larger subsystems. Defines hierarchical organization and modular decomposition.
- Architectural Style**
The **architectural style that guides this organization**: these elements and their interfaces, their collaborations, and their composition.

 Usage How system is used	 Functionality What system does	 Performance Speed and efficiency	 Resilience Fault tolerance
 Reuse Component reusability	 Comprehensibility Understandability	 Constraints Economic & technical limits	 Aesthetics Elegance and beauty

UP Management Perspective

Four Phases with Milestones



💡 Inception

Defining the scope of the project. Establish vision, identify key stakeholders, and define initial requirements. Feasibility analysis and risk assessment.

🔧 Elaboration

Planning, specifying features and designing architecture. Detailed requirements, system architecture, and development plan.

🔧 Construction

Building the product. Implementation of remaining features, testing, and preparation for deployment.

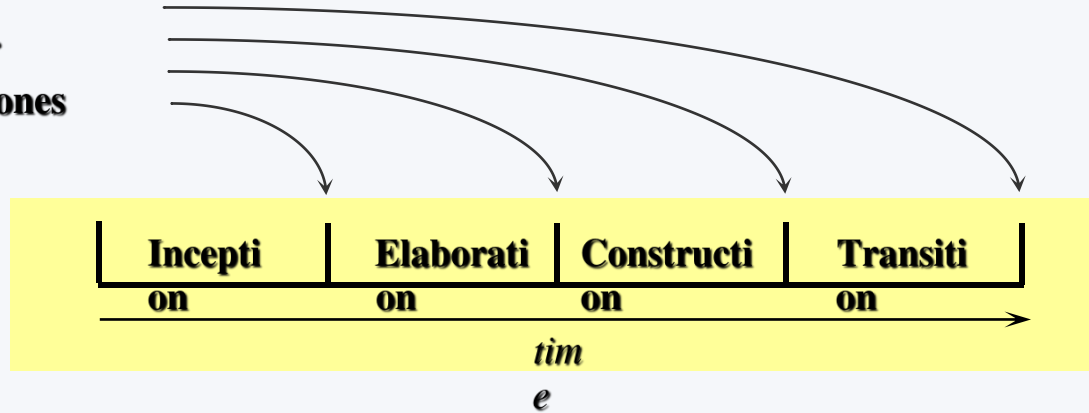
🔑 Transition

Transitioning the product into its user community. Deployment, training, and transition to maintenance.

UP Management Perspective

Development Cycles and Releases

Major milestones

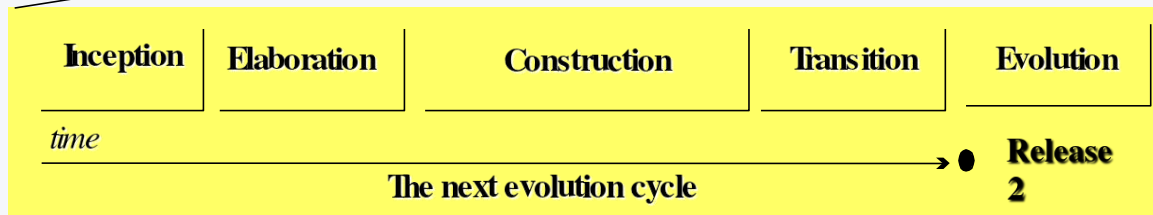
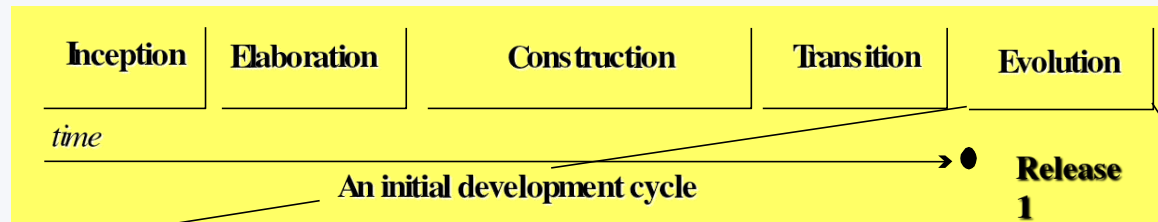


Development Cycles

A development cycle includes one execution of all four phases and produces a customer release. Successful software systems require several cycles that might be sequential or might overlap.

Cycle Characteristics

- ✓ **Sequential or Overlapping:** Cycles may run consecutively or overlap
- ✓ **Customer Releases:** Each cycle produces a working product
- ✓ **Incremental Evolution:** System grows with each cycle
- ✓ **Risk Mitigation:** Early cycles address highest risks



Multiple Releases

- Release 1: Core features
- Release 2: Extended features
- Release 3: Advanced features
- Release N: Full system

UP Engineering Perspective

Iterations and Executable Releases

What is an Iteration?

Development Activities

An iteration encompasses the **development activities** which produce a **release of an executable product**. It includes all workflows: requirements, analysis, design, implementation, and testing.

Subset of Complete Vision

A release can be a **subset of the complete vision**, but it must be useful from an engineering or user perspective. Each iteration delivers working functionality.

Incremental Evolution

The development cycle is viewed as a **series of iterations through which the software incrementally evolves**. Each iteration builds upon previous ones, adding new features and refinements.

Time-boxed

Iterations typically last **2-6 weeks** with fixed duration and scope commitment.

Fixed timeline provides predictability and rhythm

Executable

Each iteration produces **working, testable software** that demonstrates tangible progress.

Reduces risk through early validation

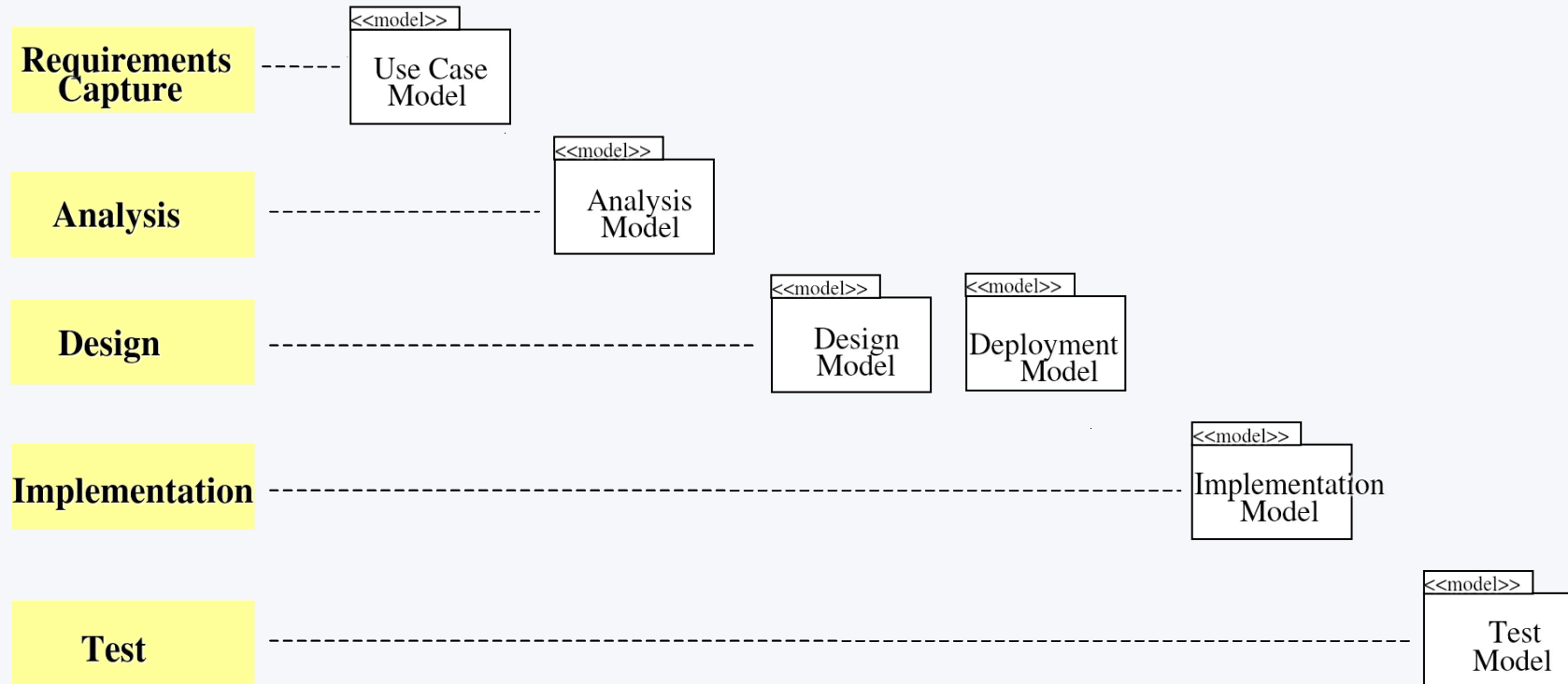
Incremental

Builds upon previous iterations, **adding new functionality** and refining existing features.

Evolutionary approach to development

UP Engineering Perspective

Models and Their Relationships

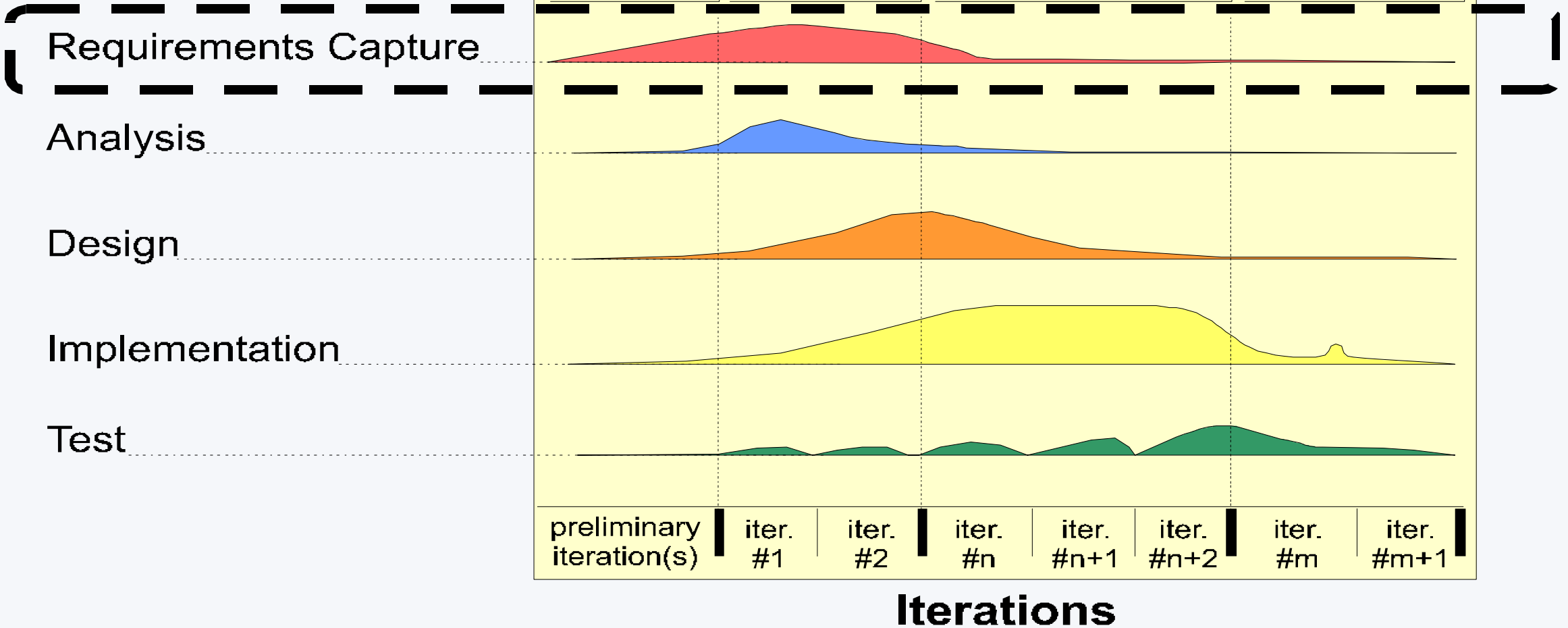


Model Traceability

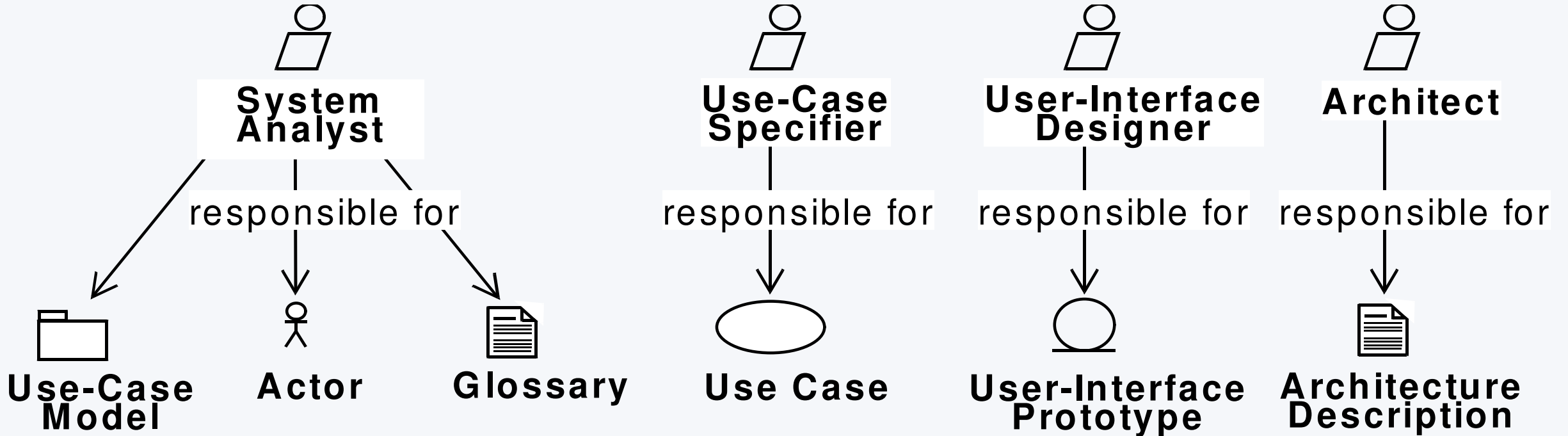
Each workflow produces a model that traces to previous models. Use Case Model → Analysis Model → Design Model → Implementation Model → Test Model → Deployment Model. This traceability ensures consistency and enables impact analysis of changes.

Requirements Capture

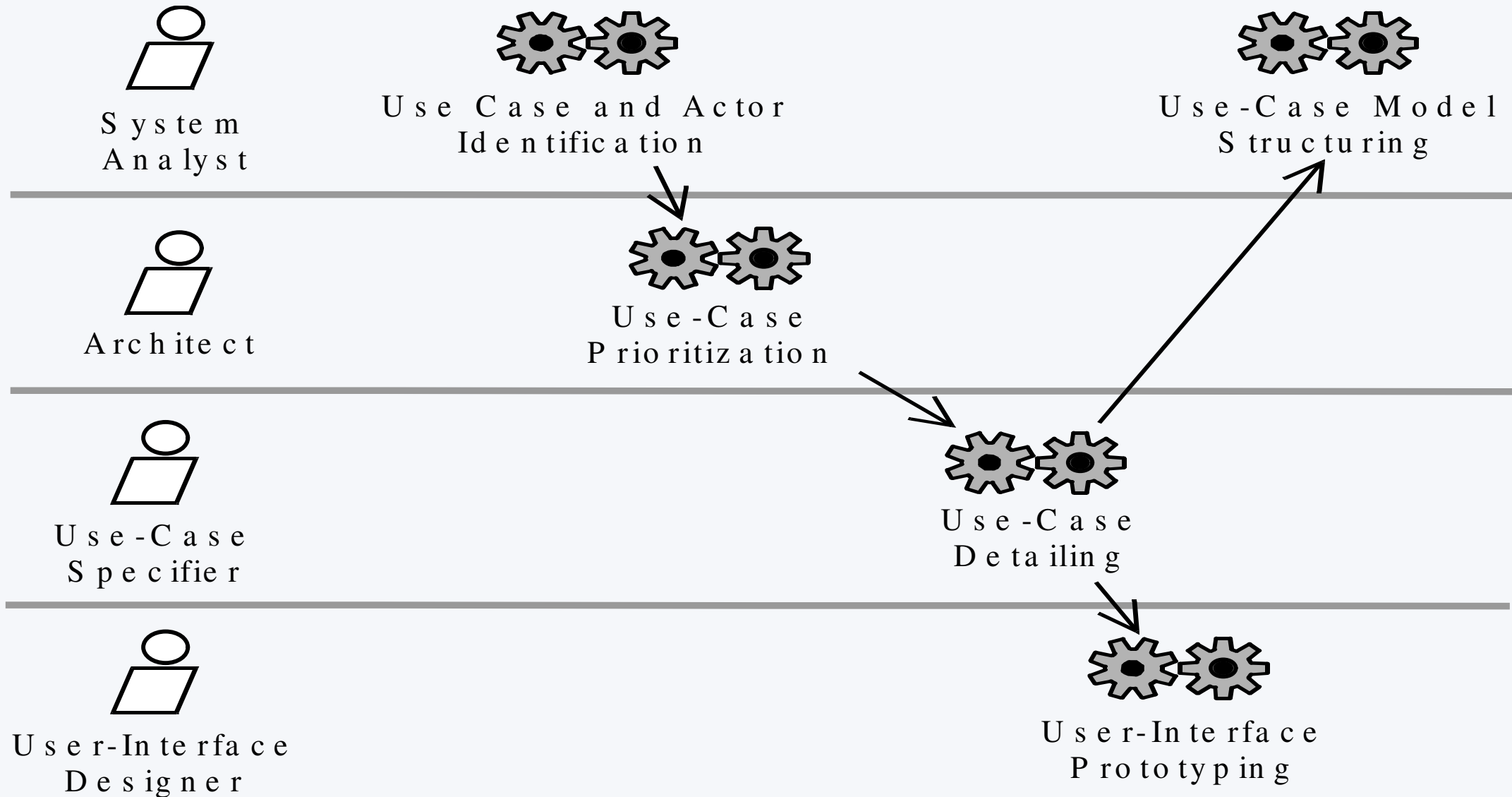
Core Workflows



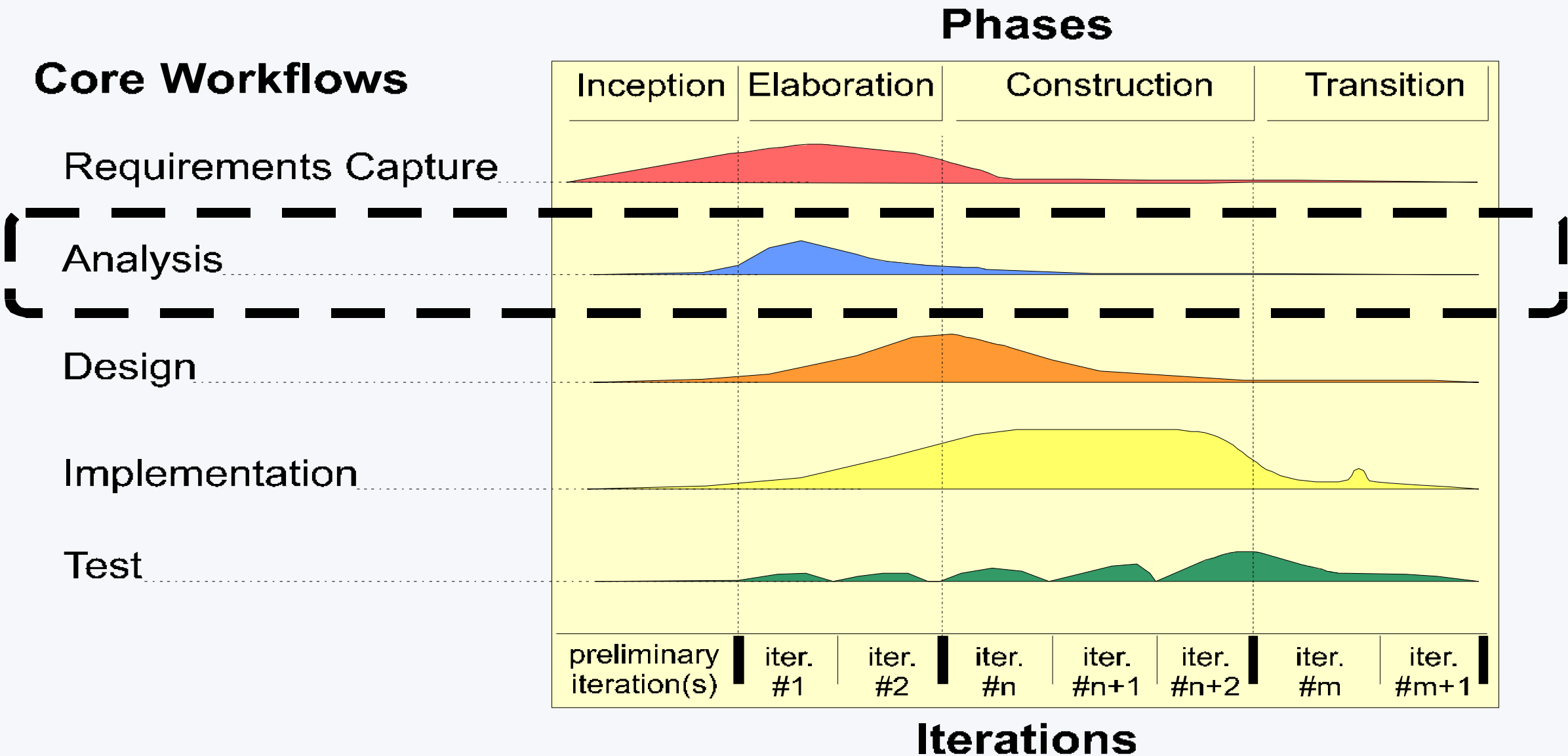
Workers and Responsibilities



Workflow in Requirements Capture



Analysis



Analysis Model

Purpose and Objectives

1 Deep Understanding

Get a deeper understanding of the functional requirements by analyzing use cases in detail. Clarifies what the system must do from user perspective.

Detailed examination of user needs and system behavior

2 Precise Specification

By getting a precise specification of the use cases, this specification becomes the analysis model. Formalizes requirements in unambiguous terms.

Formal documentation of system requirements

3 Structured Specification

By structuring them, that is the specification, to facilitate preparing it and understanding it, and to make it easy to change. Organizes requirements logically.

Well-organized, maintainable requirements

4 Logical Structure

To specify a logical structure of the system that will make it easy to understand, resilient, robust and changeable for the life of the system. Creates stable foundation.

Stable, maintainable architecture foundation

5 Drive Development Activities

To create a specification that drives design, implementation and testing activities. The analysis model serves as blueprint for subsequent development phases.

Analysis

Class Types: Boundary, Control, Entity

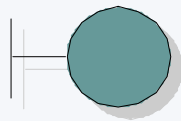


Boundary Class

Models communication between the actors and the system's inner workings. Represents the interface between system and external world.

Examples:

- User interface screens
- API endpoints
- System interfaces
- External service adapters



Boundary

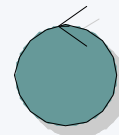


Control Class

Models control behavior specific to one or a few use cases. Coordinates the activities of other classes and implements business logic.

Examples:

- Transaction managers
- Workflow coordinators
- Business rule engines
- Session controllers



Control

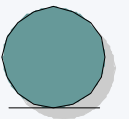


Entity Class

Models information and associated behavior about some phenomenon. Represents persistent data and business objects.

Examples:

- Customer, Account, Product
- Order, Invoice, Payment
- Course, Student, Enrollment
- Persistent business objects



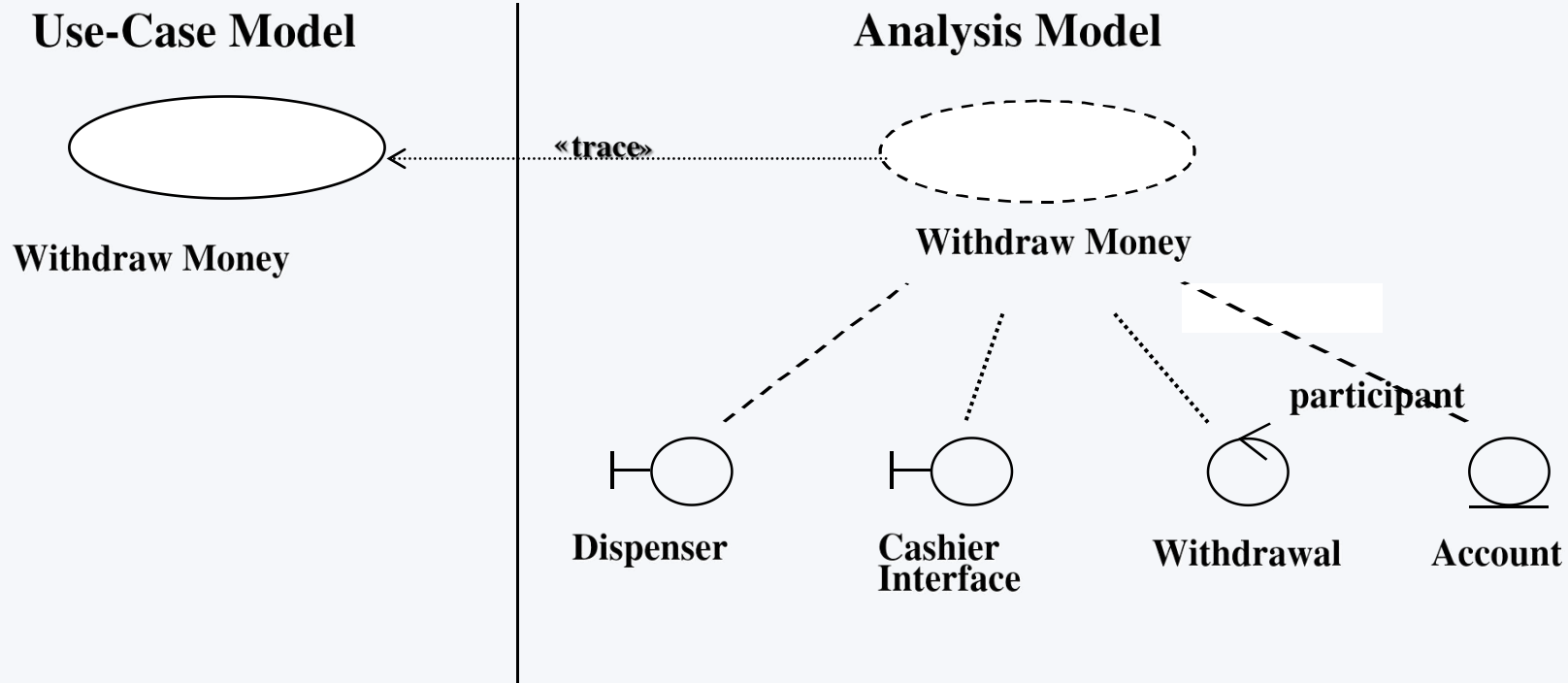
Entity

Separation of Concerns

These three class types follow the **Model-View-Controller (MVC)** pattern. **Boundary** classes handle presentation (View), **Control** classes manage application logic (Controller), and **Entity** classes represent data (Model). This separation improves maintainability and flexibility.

Analysis

Class Types: Boundary, Control, Entity

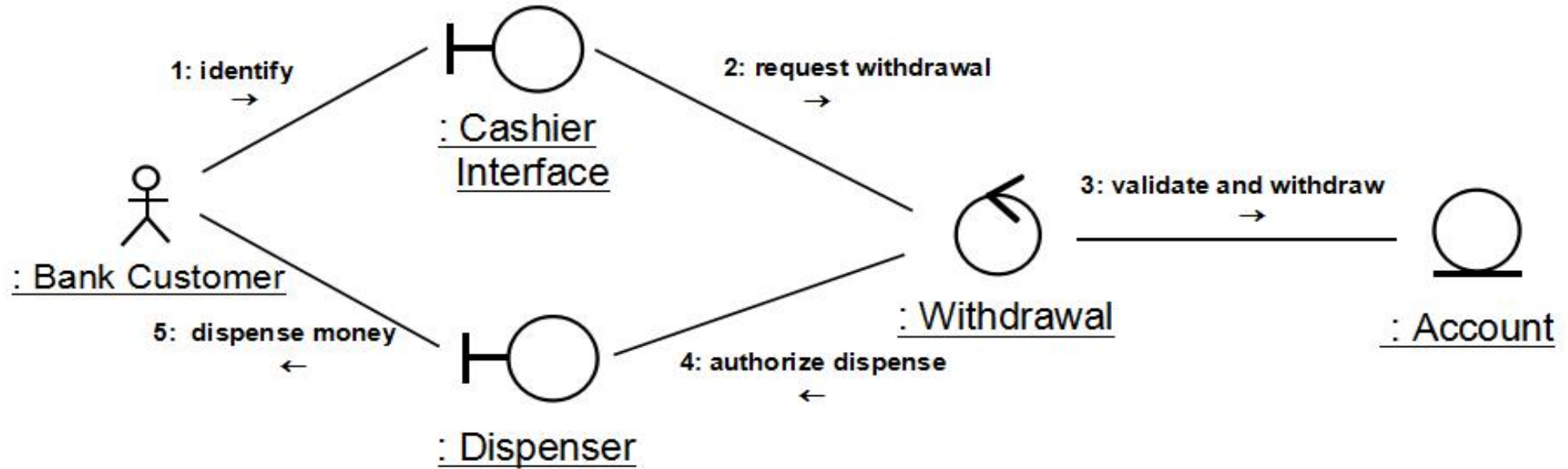


Separation of Concerns

These three class types follow the **Model-View-Controller (MVC)** pattern. **Boundary** classes handle presentation (View), **Control** classes manage application logic (Controller), and **Entity** classes represent data (Model). This separation improves maintainability and flexibility.

Use-Case Realizations Analysis

Class Types: Boundary, Control, Entity



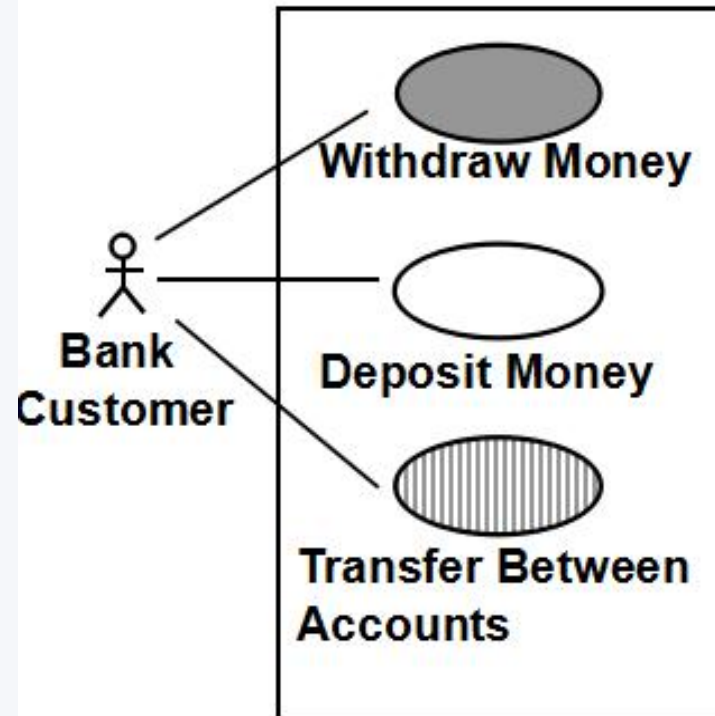
Separation of Concerns

These three class types follow the **Model-View-Controller (MVC)** pattern. **Boundary** classes handle presentation (View), **Control** classes manage application logic (Controller), and **Entity** classes represent data (Model). This separation improves maintainability and flexibility.

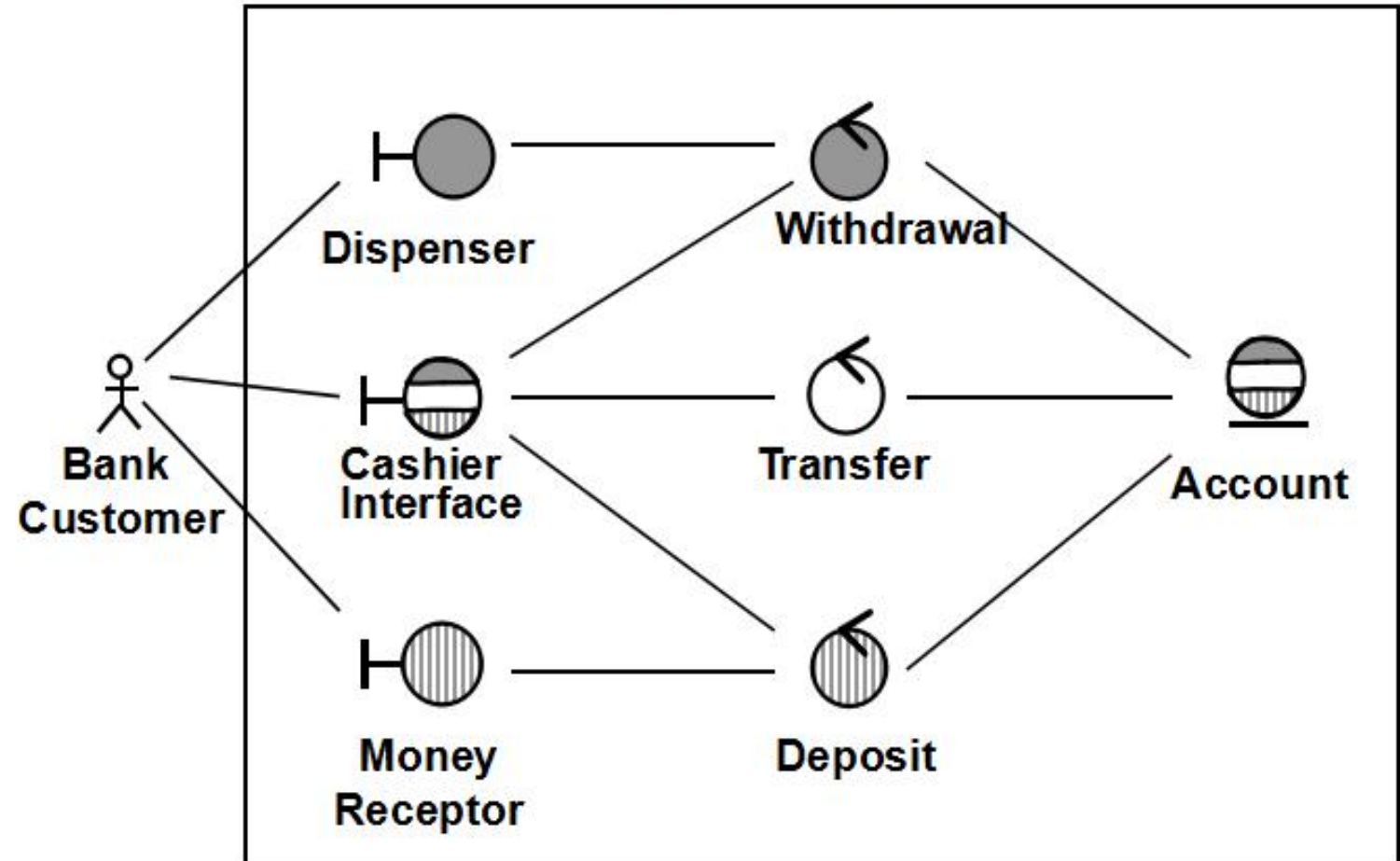
Use-Case Realizations Analysis

Class Types: Boundary, Control, Entity

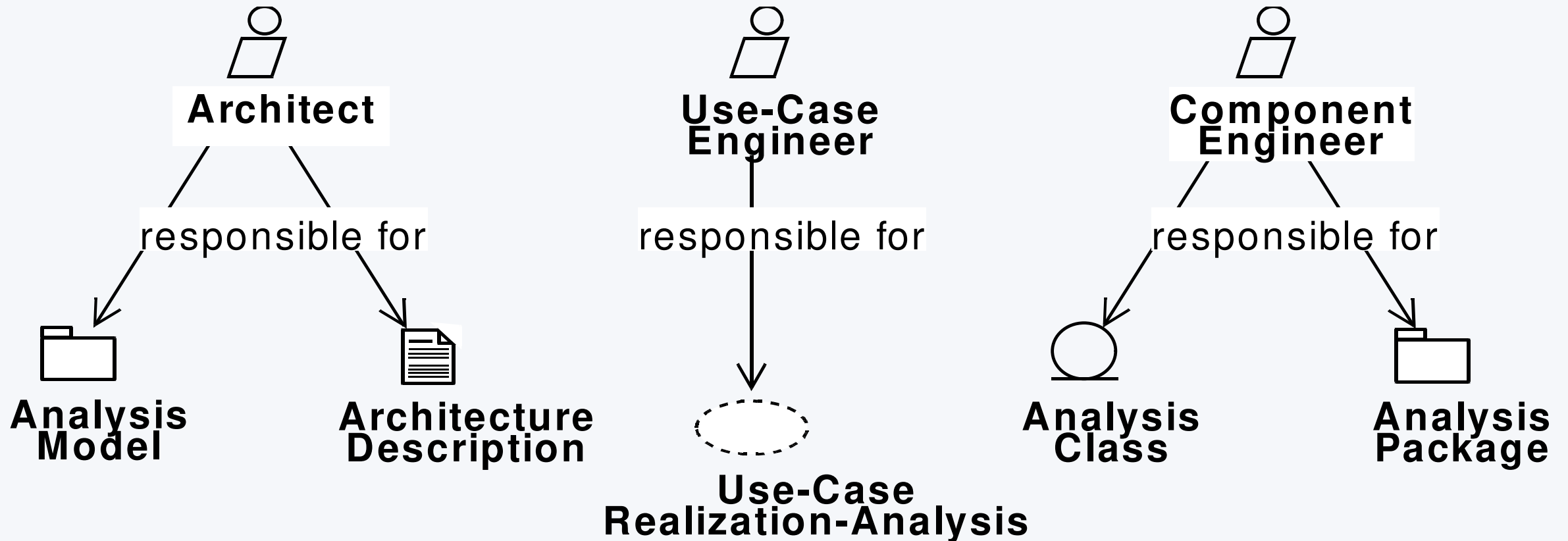
Use-Case Model



Analysis Model



Workers and Responsibilities



Analysis Workflow



Architect



Architectural
Analysis



Use-Case
Engineer



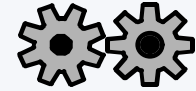
Use-Case
Analysis



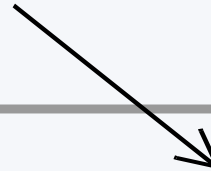
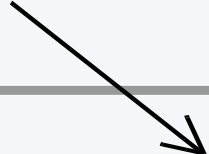
Component
Engineer



Class
Analysis

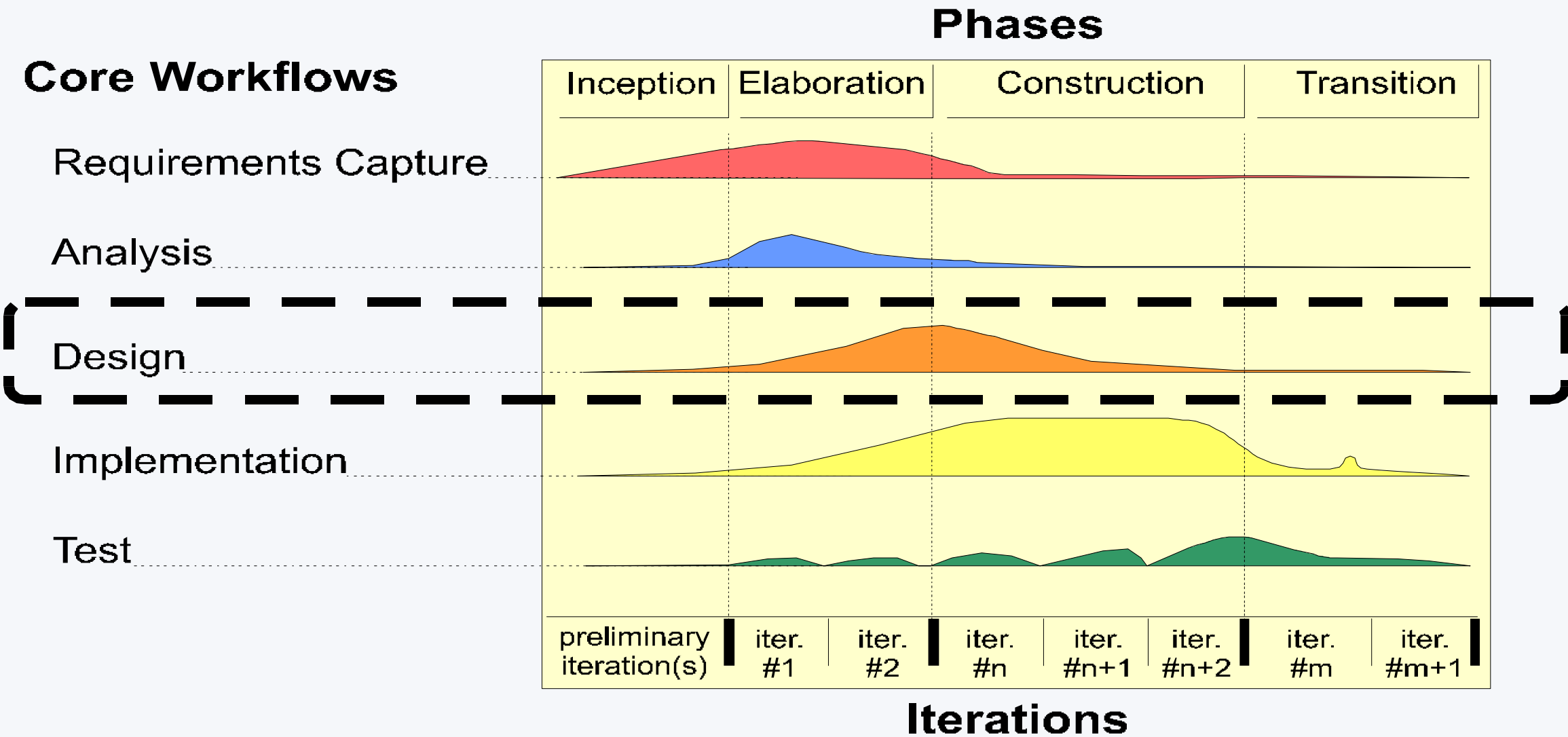


Package
Analysis



Design Model

Core Workflows



Design Model

Purpose and Objectives

Design Model Objectives

Build Stable Architecture

To build a system with a stable architecture by balancing the logical architecture from analysis with nonfunctional requirements. Creates robust, resilient system structure.

Balance Multiple Factors

Balance analysis architecture with programming languages, component reuse, operating systems, distribution and concurrency technologies, database technologies, user-interface technologies, transaction management technologies, and so on.

Implementation Blueprint

To create a blueprint of the implementation that guides developers in building the system. Provides detailed specifications for coding activities.



Programming Languages



Component Reuse



Operating Systems



Distribution Technologies



Database Technologies



UI Technologies



Transaction Management



And More...

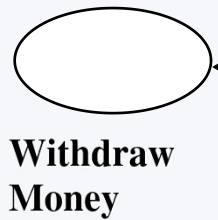
Design Model

Traceability: Analysis to Design

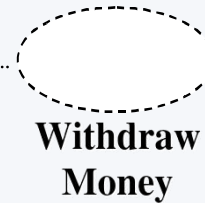
Model Traceability

Traceability Chain

Use-Case Model



Analysis Model



Design Model



«trace»

«trace»

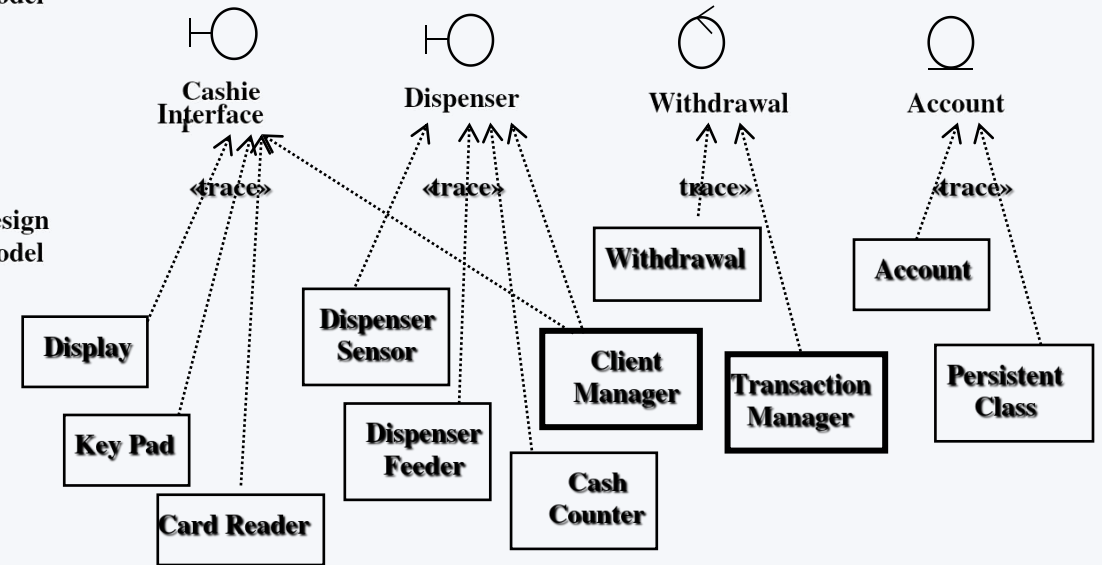
Benefits

- Change impact analysis
- Requirements coverage
- Design validation
- Consistency checking

ATM Example: Class Evolution

Analysis Model

Design Model

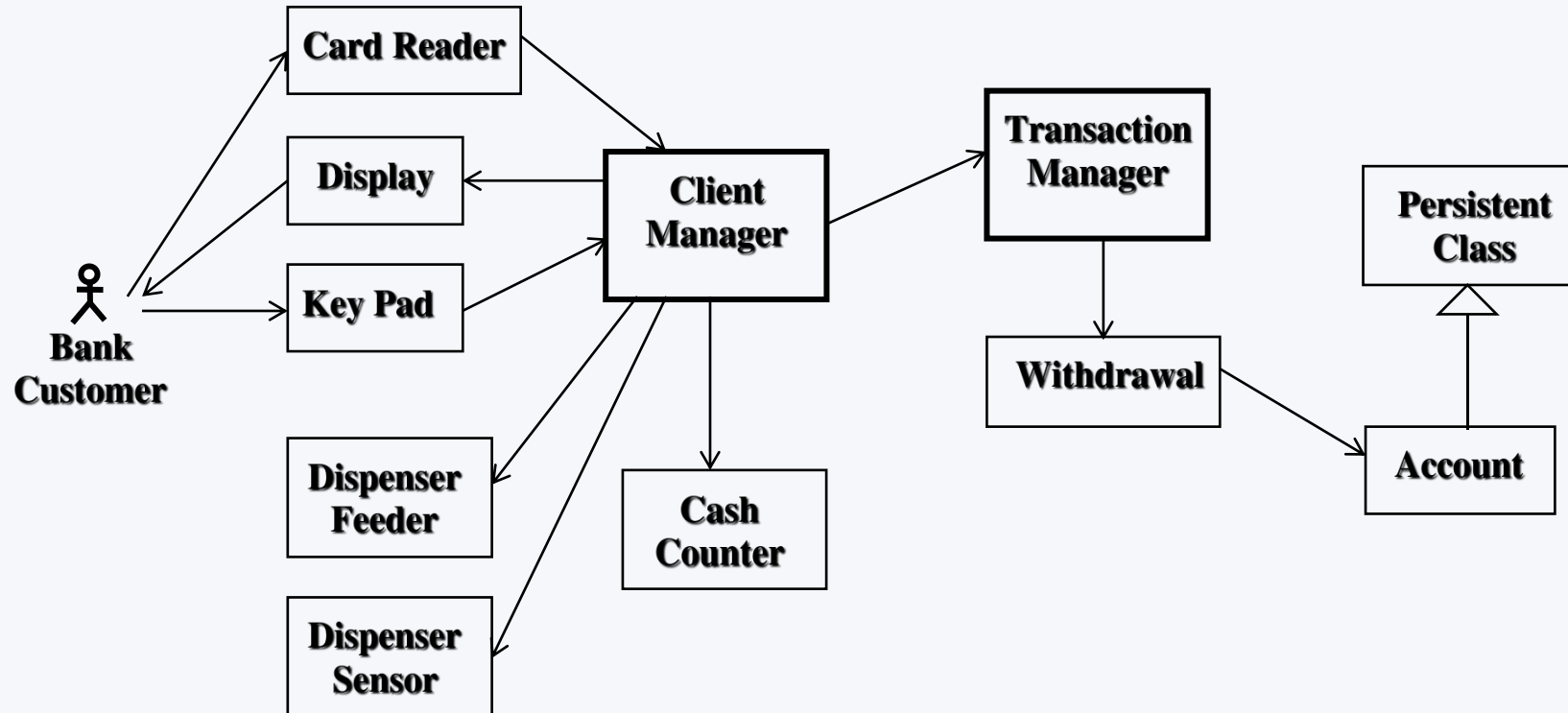


Refinement Process: Analysis classes are refined into design classes that consider implementation technologies, performance requirements, and deployment constraints.

Design Model

Class Diagrams: Static Structure

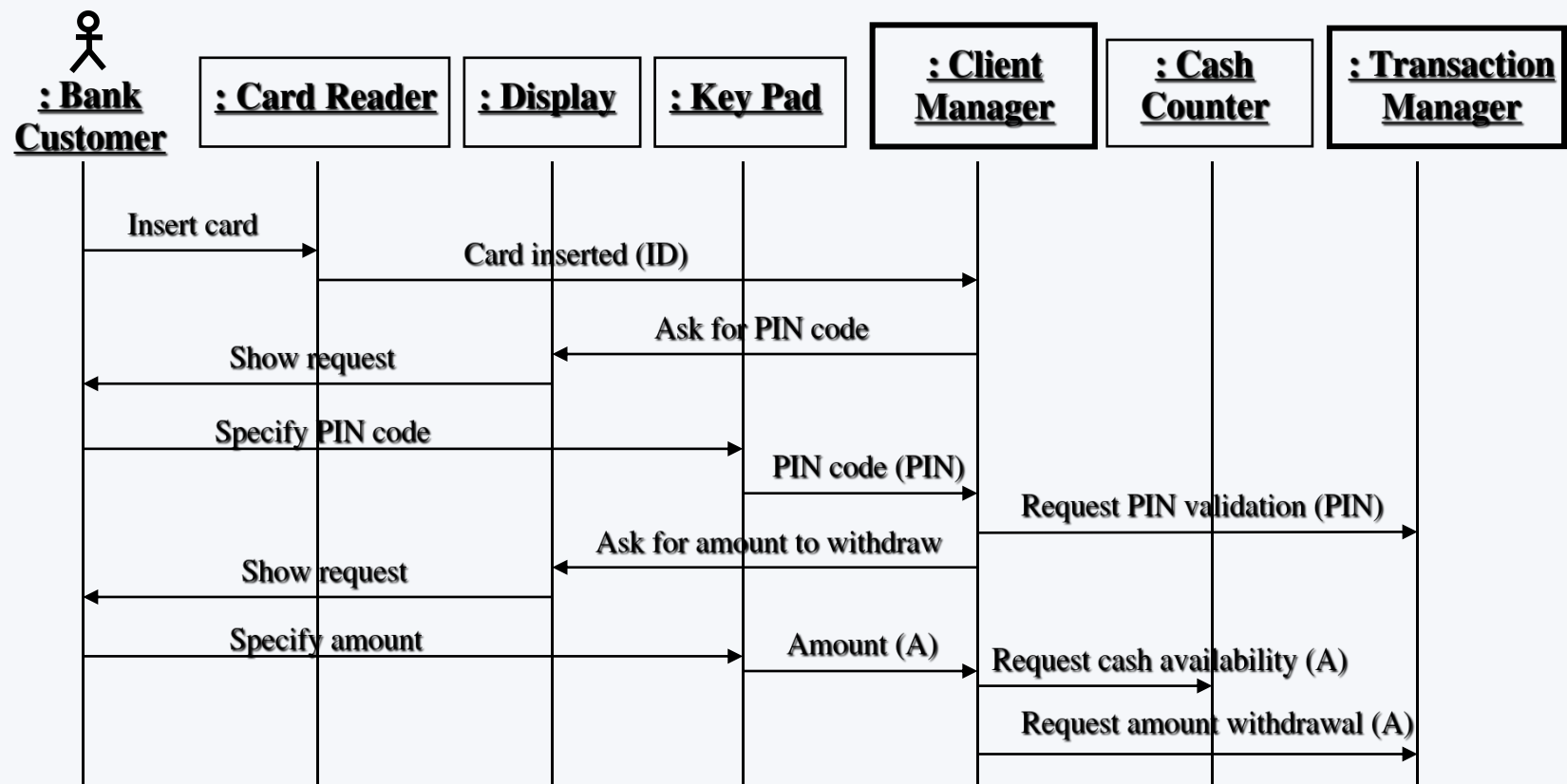
ATM System Design Classes



Design Model

Sequence Diagrams: Dynamic Interactions

ATM Withdrawal Sequence Flow



Dynamic Behavior

Object interactions over time



9+ Objects

Collaborating in sequence



Messages

Method calls between objects

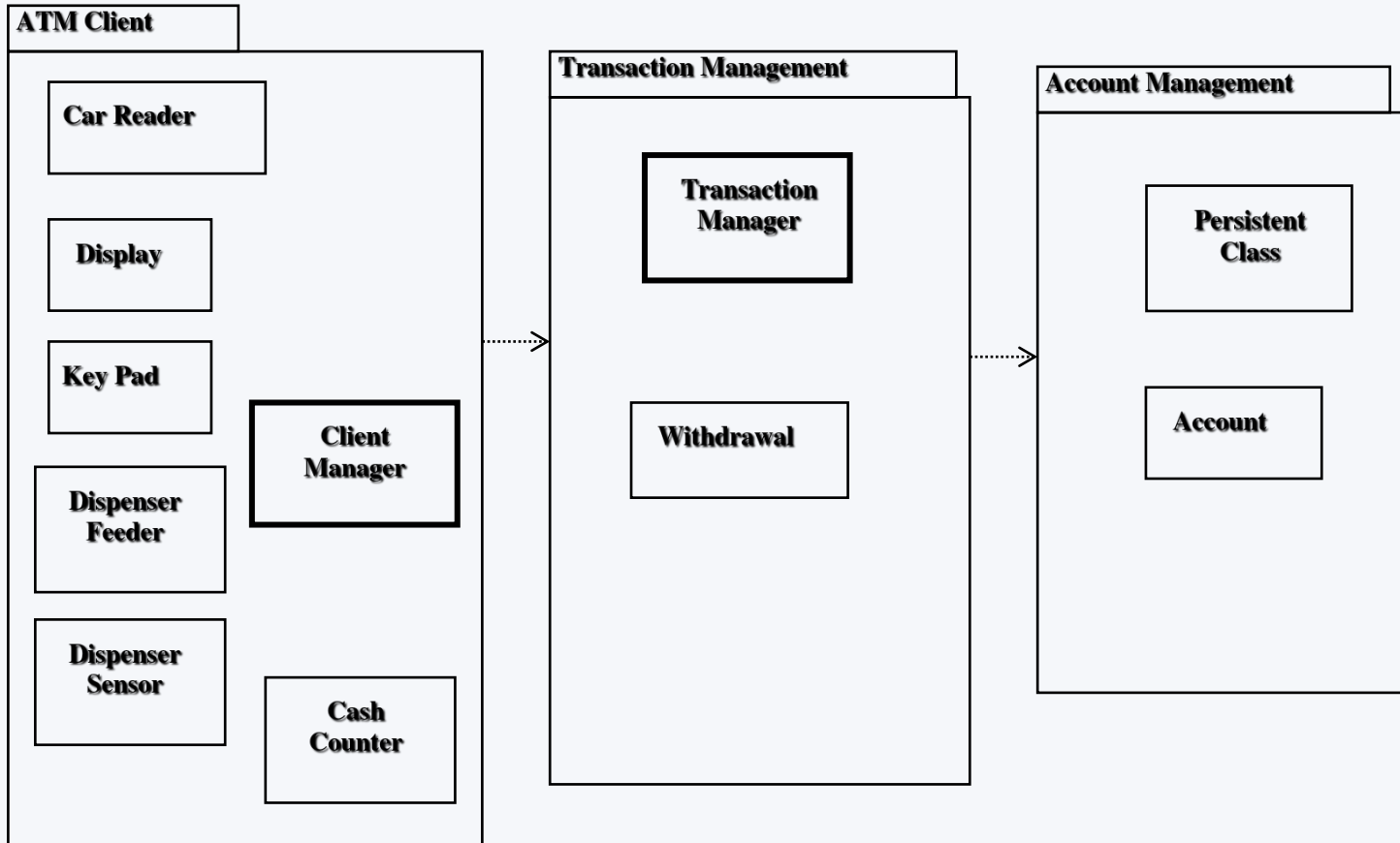


Time-ordered

Top to bottom execution

Design Model

Package Diagram: System Organization



Package Organization

Package diagram shows system organization into **logical subsystems** that group related classes and components. Each package represents a cohesive set of responsibilities.

ATM Subsystems

-  **ATM Client:** User interface and client logic
-  **Transaction Management:** Transaction processing
-  **Account Management:** Account services

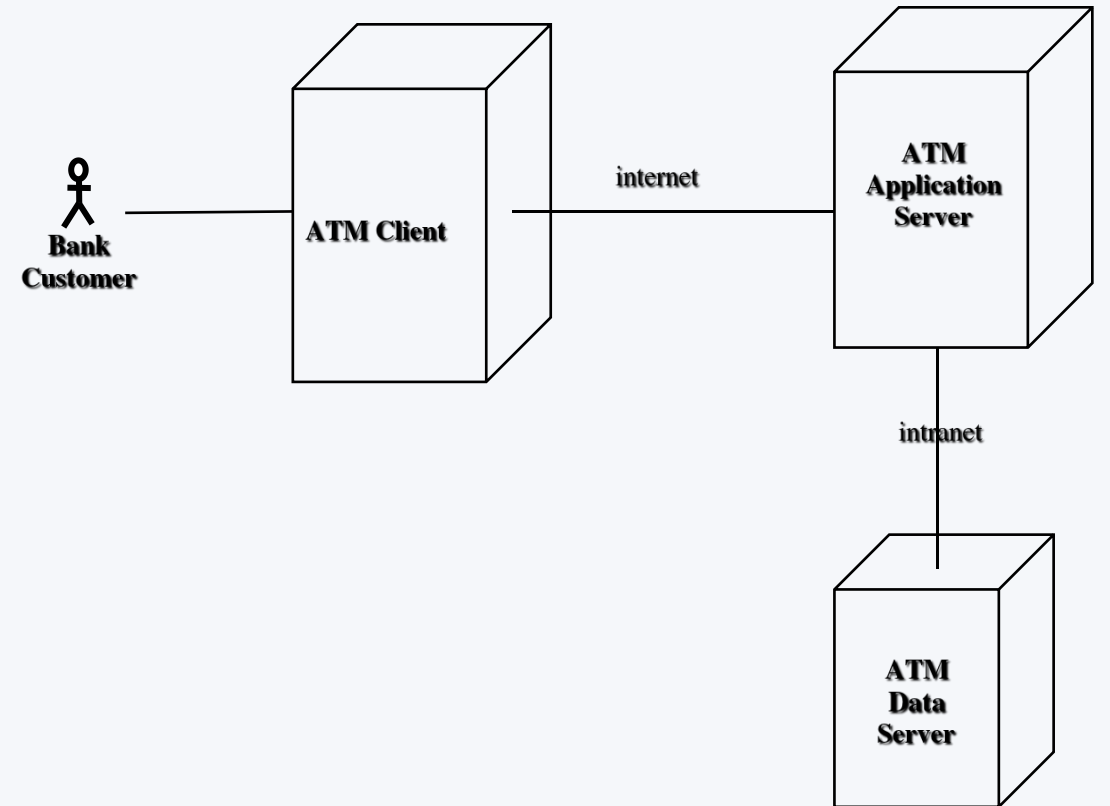
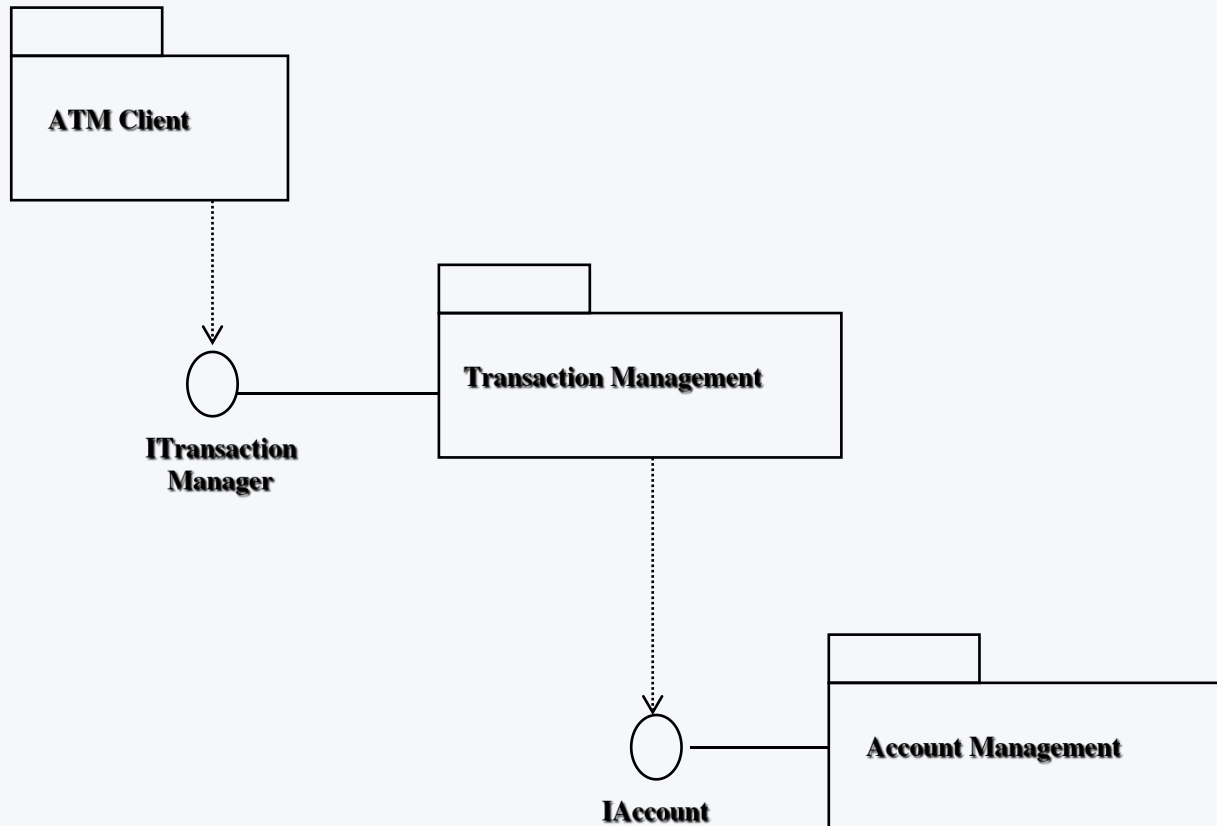
Dependencies

- Client depends on Transaction and Account
- Transaction depends on Account
- Dispenser is independent

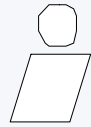
Design Model

Deployment Diagram: Physical Architecture

ATM System Subsystems , interfaces and Physical Architecture

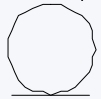


Workers and Responsibilities



Architect

responsible for



Design Model

Deployment
Model

Architecture
Description



Use-Case
Engineer

responsible for

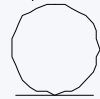
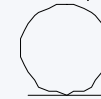


Use-Case Realization
- Design



Component
Engineer

responsible for

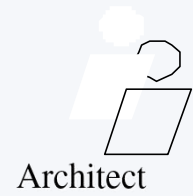


Design Class

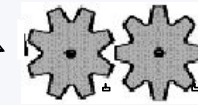
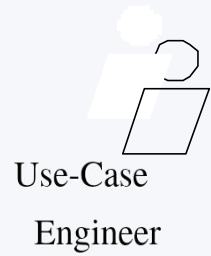
Design
Subsystem

Interface

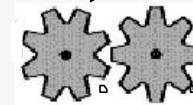
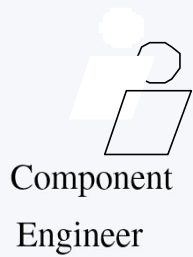
Design Workflow



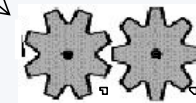
Architectural Design



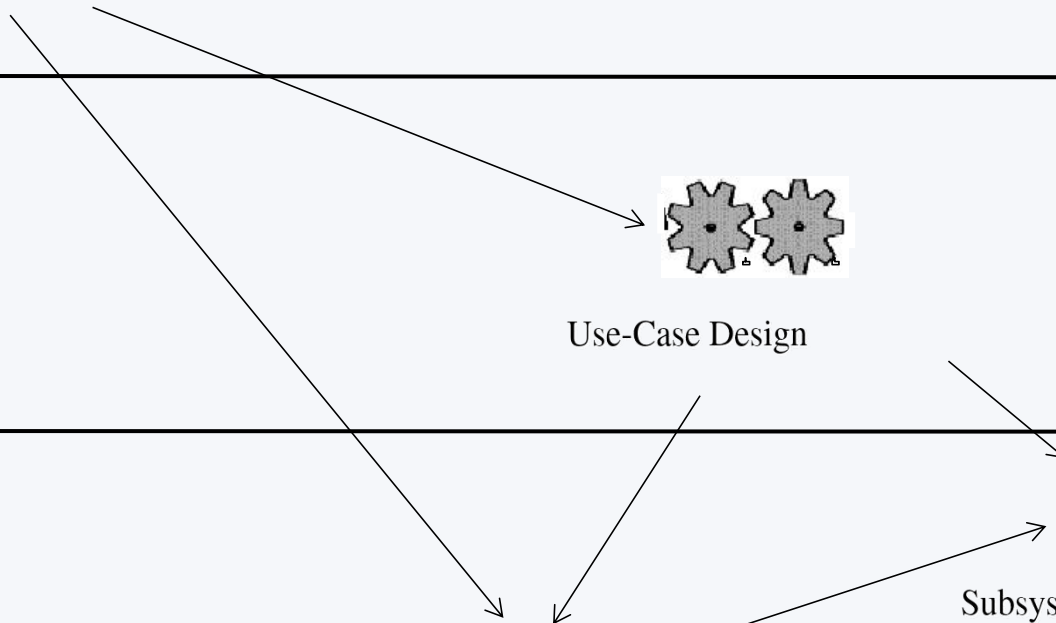
Use-Case Design



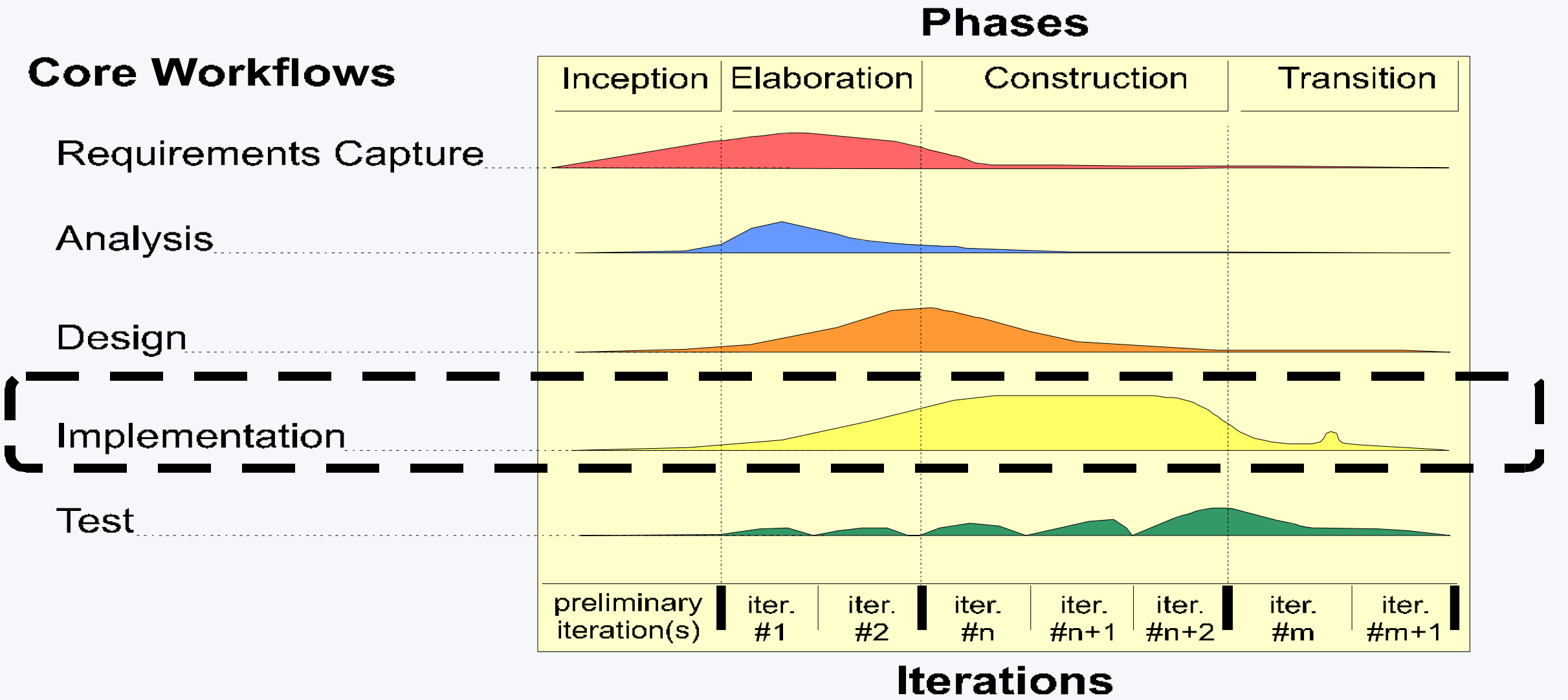
Class Design



Subsystem Design



Implementation Model



Implementation Model

Code Organization and Component Mapping

Implementation Activities

Code Organization

Define the organization of the code, in terms of implementation subsystems organized in layers. Establish module structure and dependency relationships.

Component Implementation

Implement classes and objects in terms of components (source files, binaries, executables, and others). Transform design specifications into executable code.

Unit Testing

Test the developed components as units to verify individual functionality and identify defects early. Execute white-box and black-box tests.

System Integration

Integrate the results produced by individual implementers (or teams), into an executable system. Combine components and resolve integration issues.



Subsystems

Organized in layers



Source Files

Implementation code



Binaries

Compiled components



Executables

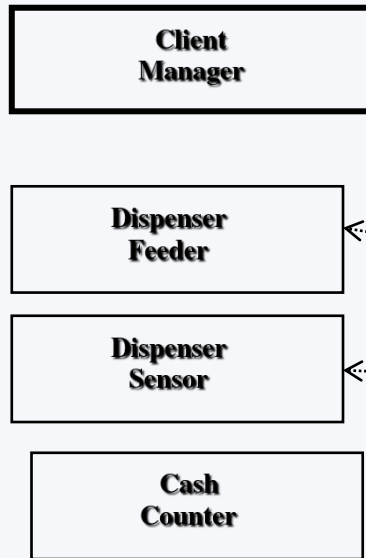
Deployable units

Implementation Model

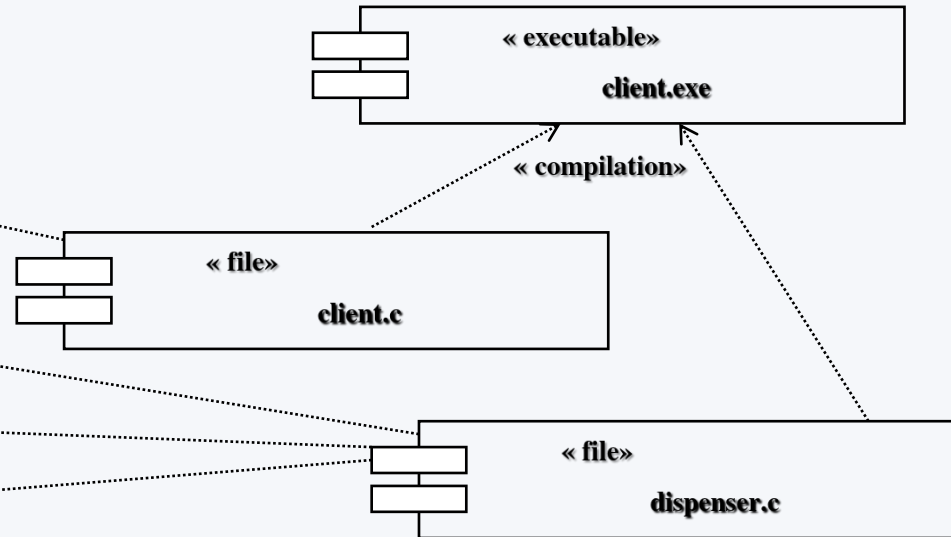
Component Mapping: Design to Executable

Design to Implementation Mapping

Design Model



Implementation Model



« trace »

« compilation »

« trace »

</>Compilation

Source files (*.c, *.cpp, *.java) are compiled into executable components (*.exe, *.dll, *.jar) through the compilation process.

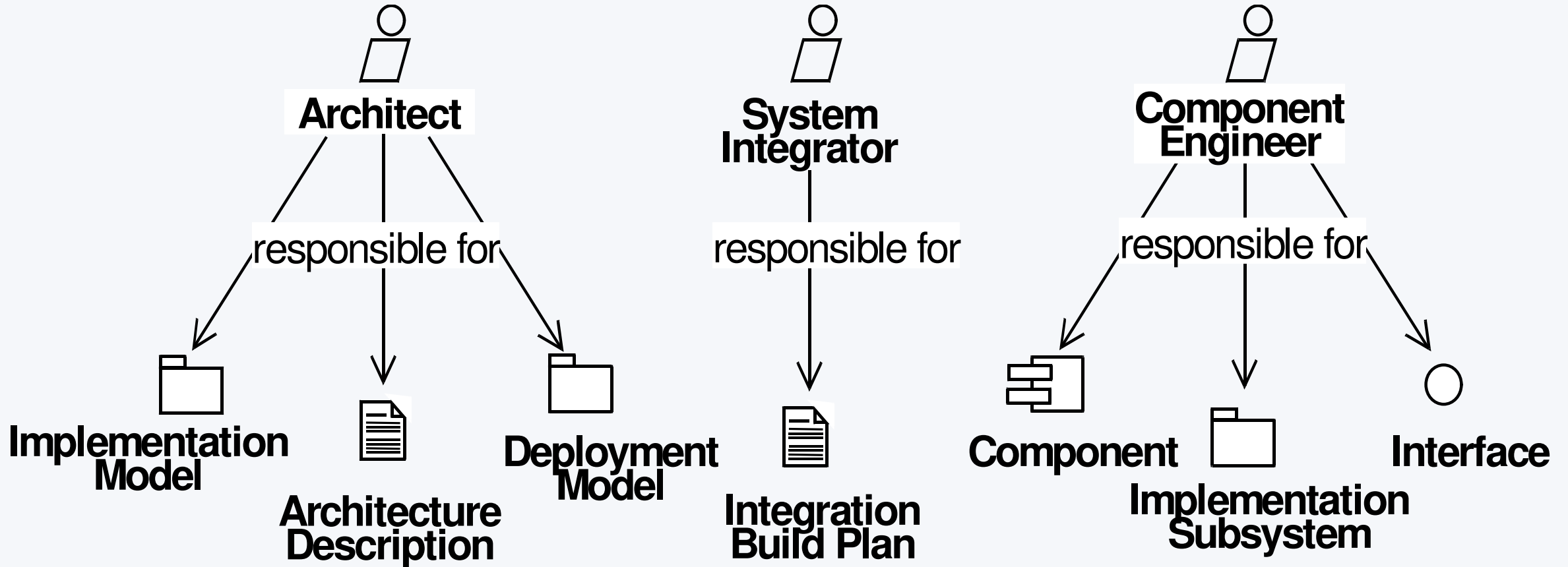
Traceability

Implementation Model maintains **traceability links** back to Design Model, enabling impact analysis and requirement validation.

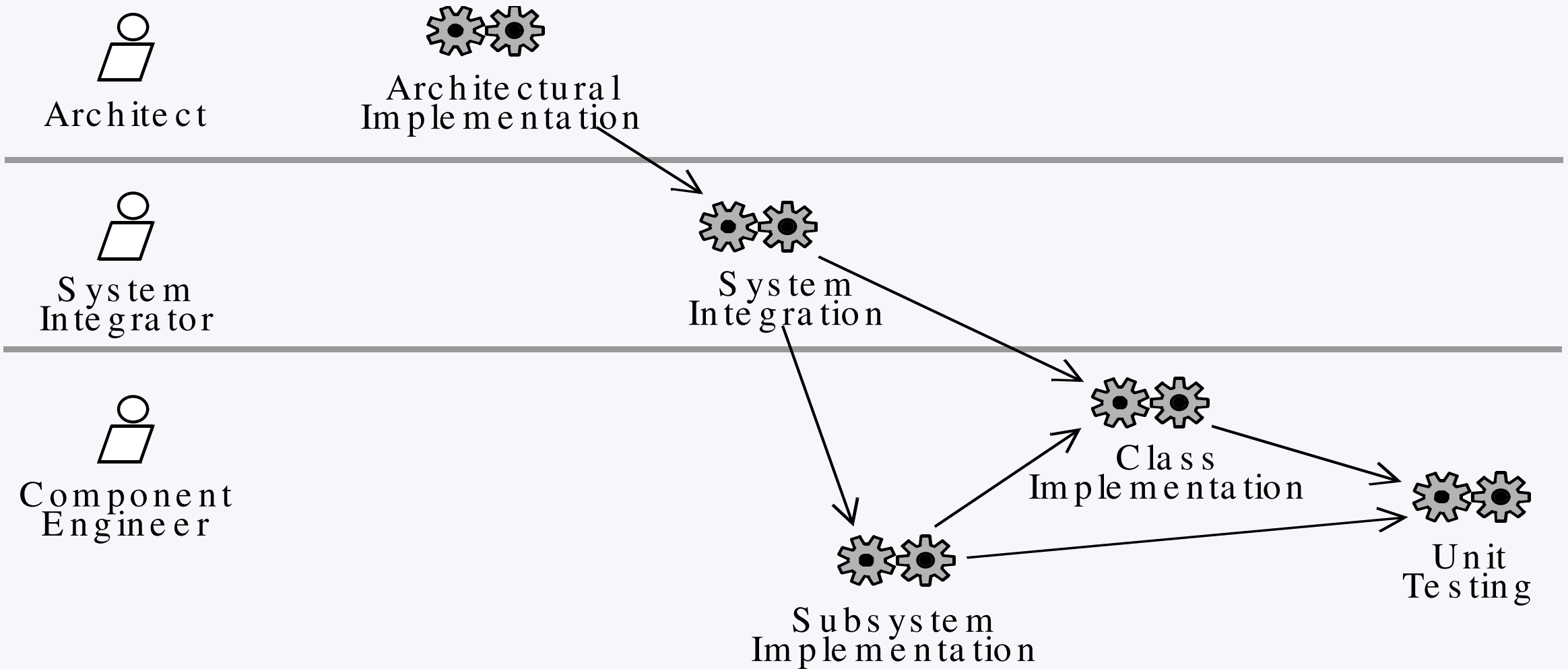
Deployment

Components can be **deployed onto nodes** as specified in the Deployment Model, creating the runtime system.

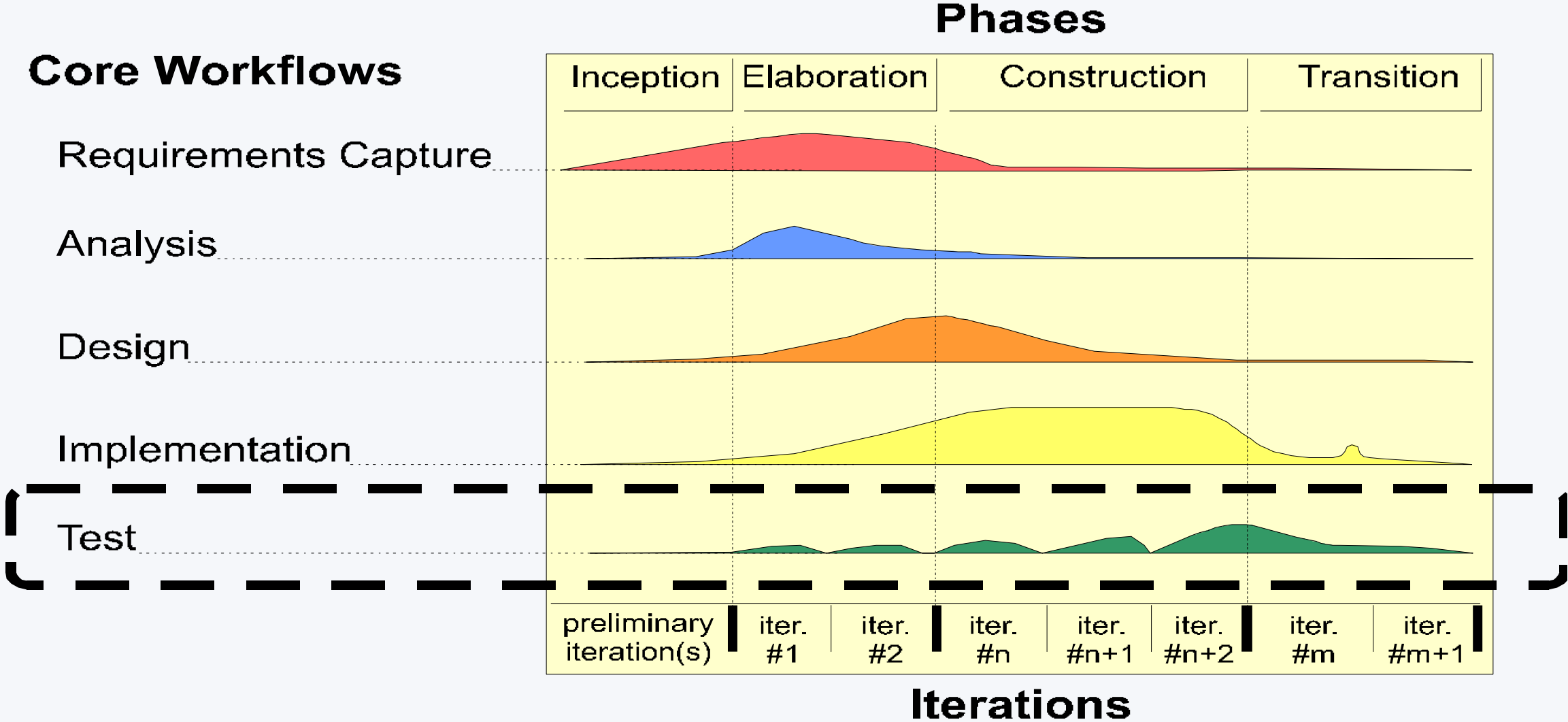
Workers and Responsibilities



Implementation Workflow



Test Model



Test Model

Overview and Objectives

What Does Testing Verify?



Object Interaction

To **verify the interaction between objects** and ensure that messages are passed correctly between system components. Validates dynamic behavior.



Component Integration

To **verify the proper integration of all components** of the software. Ensures that individually working components function together correctly.



Requirements Implementation

To **verify that all requirements have been correctly implemented** and the system delivers the expected functionality. Validates against specifications.



Defect Identification

To **identify and ensure defects are addressed** prior to the deployment of the software. Finds and fixes issues before they reach users.



Test Model Foundation

This is achieved by developing a test model based on: **Results from requirements capture, Results from analysis & design, Results from implementation.** The test model provides comprehensive coverage of all aspects of the system.

Test Model

Test Types and Levels

Unit Test

To verify a component or a group of **components**. Tests individual classes, methods, or modules in isolation to ensure they work correctly.

- White-box testing
- Method-level validation
- Boundary value analysis
- Statement coverage

Integration Test

To piece by piece verify implementation **subsystems as they are integrated**. Tests interactions between integrated components.

- Incremental integration
- Interface testing
- Stub and driver usage
- Top-down and bottom-up

System Test

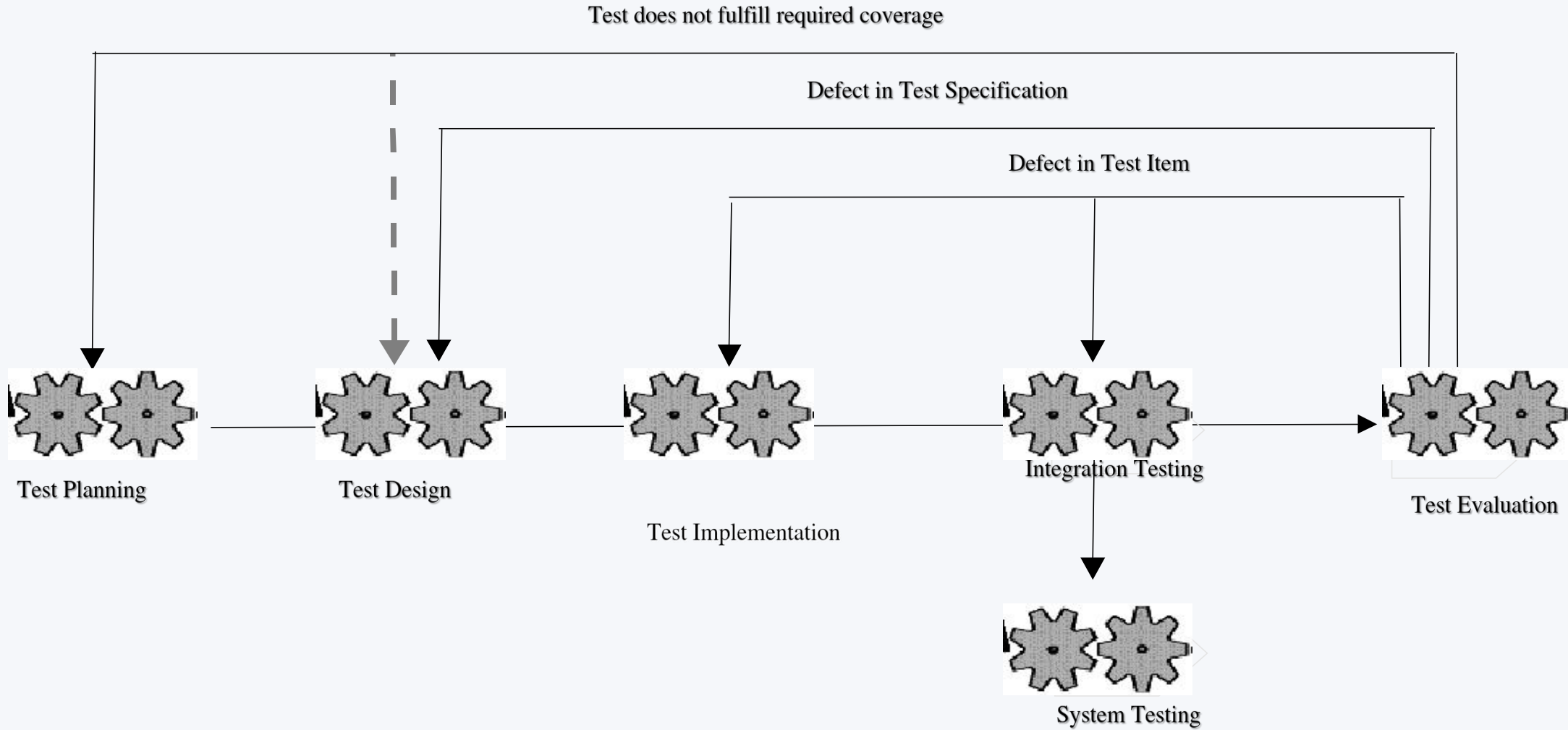
To verify the **whole system** against requirements. End-to-end testing of complete, integrated system in realistic environment.

- Black-box testing
- Use case validation
- Performance testing
- User acceptance testing

Testing Pyramid

Most tests should be **unit tests** (70-80%) as they are fast, reliable, and find defects early. **Integration tests** verify component interactions (15-25%). **System tests** validate end-to-end functionality (5-10%) but are slower and more expensive.

The Testing Lifecycle



Test Model

Composition of a test model

Composition of a test model

Test Case

A test case is derived from and traceable to a use case or scenario, a use-case realization, or a requirement Include

- INPUT
- OUTPUT
- CONDITION

Test Procedure

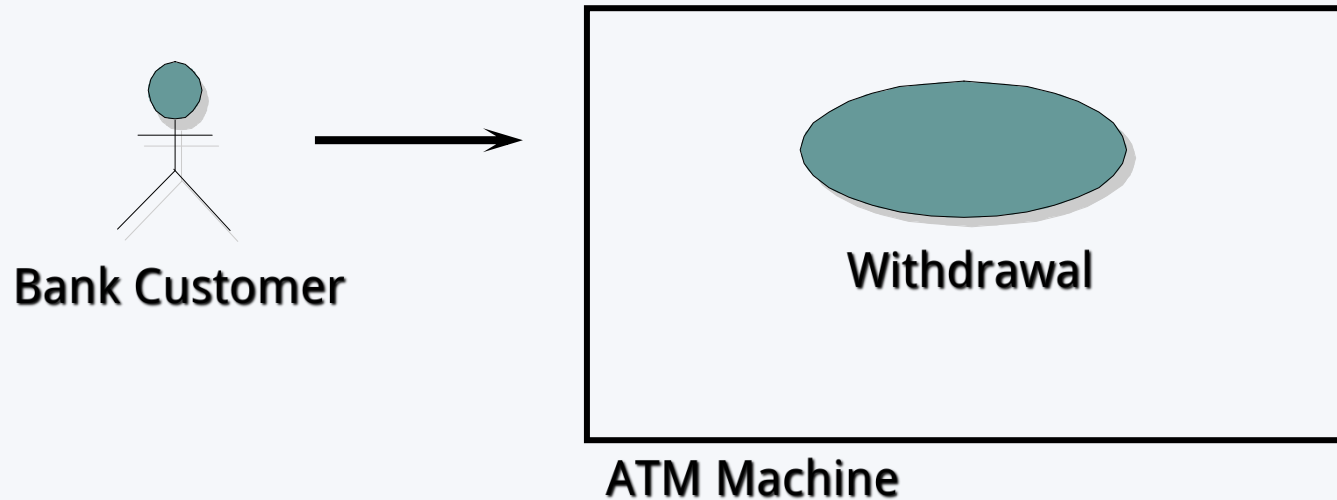
The design of a test procedure needs to address the following items

- Preconditions for the test
- Input conditions
- Actions to be taken
- Expected results
- Method for validating expected results

Test Model

Test Case Example: ATM Withdrawal

Test Case Example: ATM Withdrawal



- ATM dispenses only \$10 and \$20 bills
- Withdraw preset amounts (\$10, \$20, \$50, \$100)
- Withdraw custom amounts
- Maximum withdrawal is lesser of \$350 or amount in account
- Maximum withdrawal per day is \$350
- PIN and Account Numbers must be valid



Test Model

Test Case Example: ATM Withdrawal

Test Case Example: ATM Withdrawal

Test cases	PIN:	Account No.	Money in account?	Money in ATM?	ValuesResult
Preset amount	Yes	Yes	Yes	Yes	\$10 Rec'd \$10
Custom amount	Yes	Yes	Yes	Yes	\$240 Rec'd \$240
Invalid PIN	No	Yes	Yes	Yes	N/A Warning message
Too much requested	Yes	Yes	Yes	Yes	\$500 Warning message

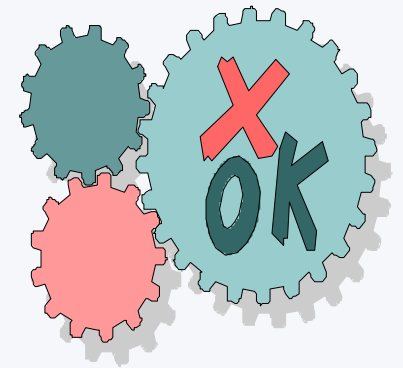
Test Model

Test Procedure Example: ATM Withdrawal

Test Procedure Example

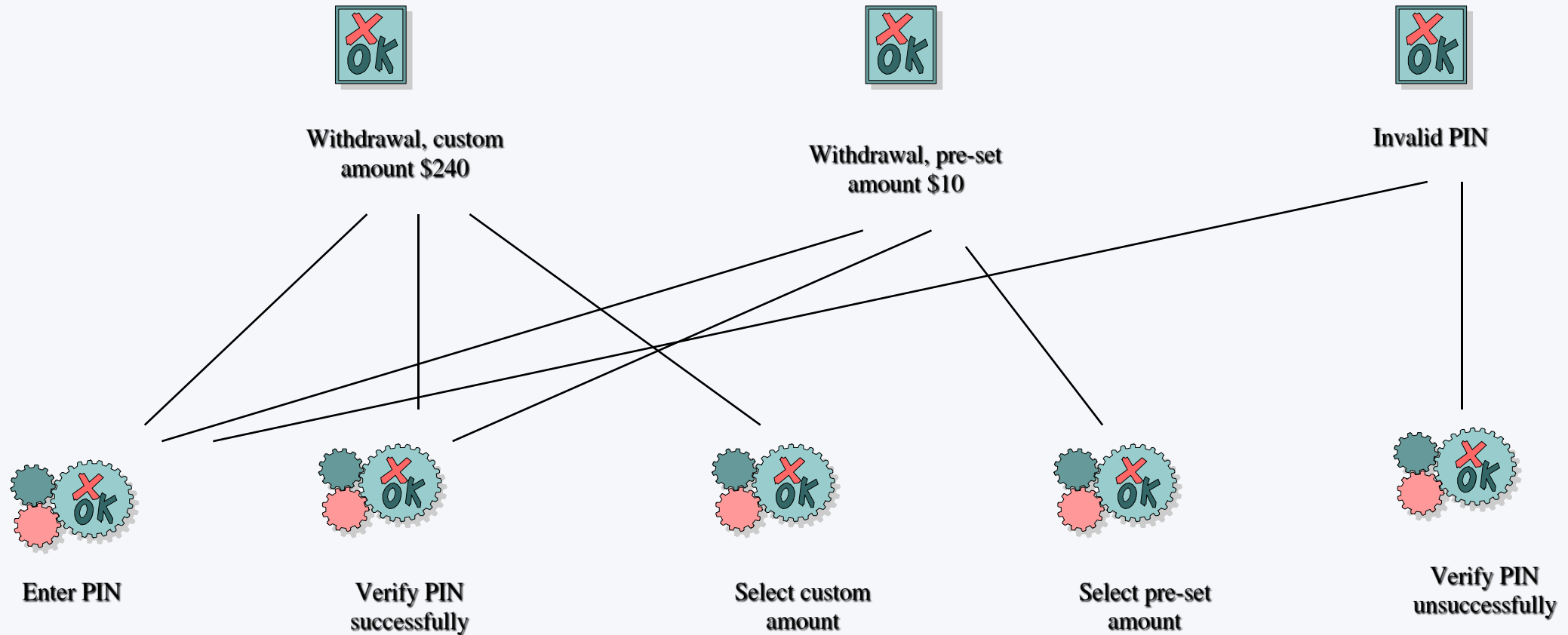
Account BK01024596 is used
PIN is valid (979695)
Account has \$1,596.75

Insert Card
Select Language (English)
Enter PIN
Wait for validation & Selection Screen
Enter Choice (pre-set)
Enter value
Wait for Cash
Wait for card and receipt
Receipt should show \$1,596.75 minus amount dispensed
Leave ATM

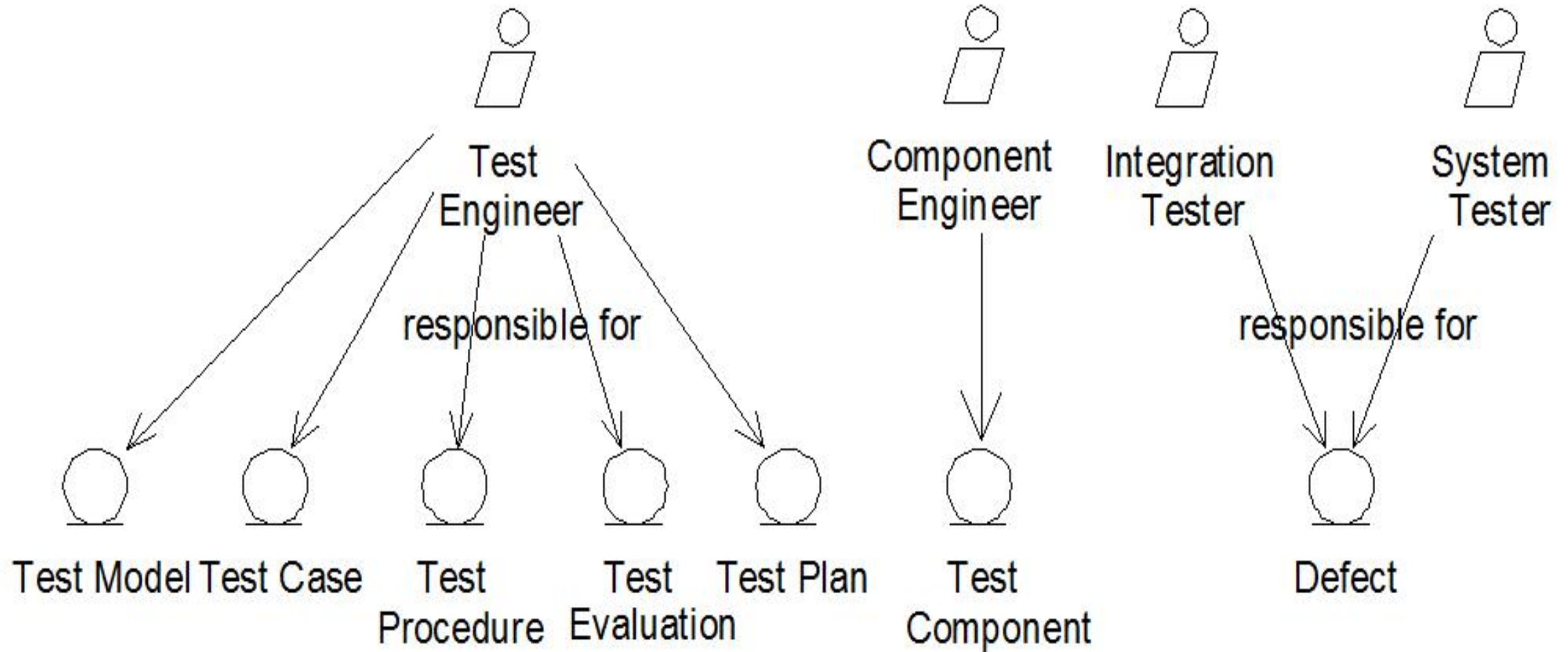


Test Model

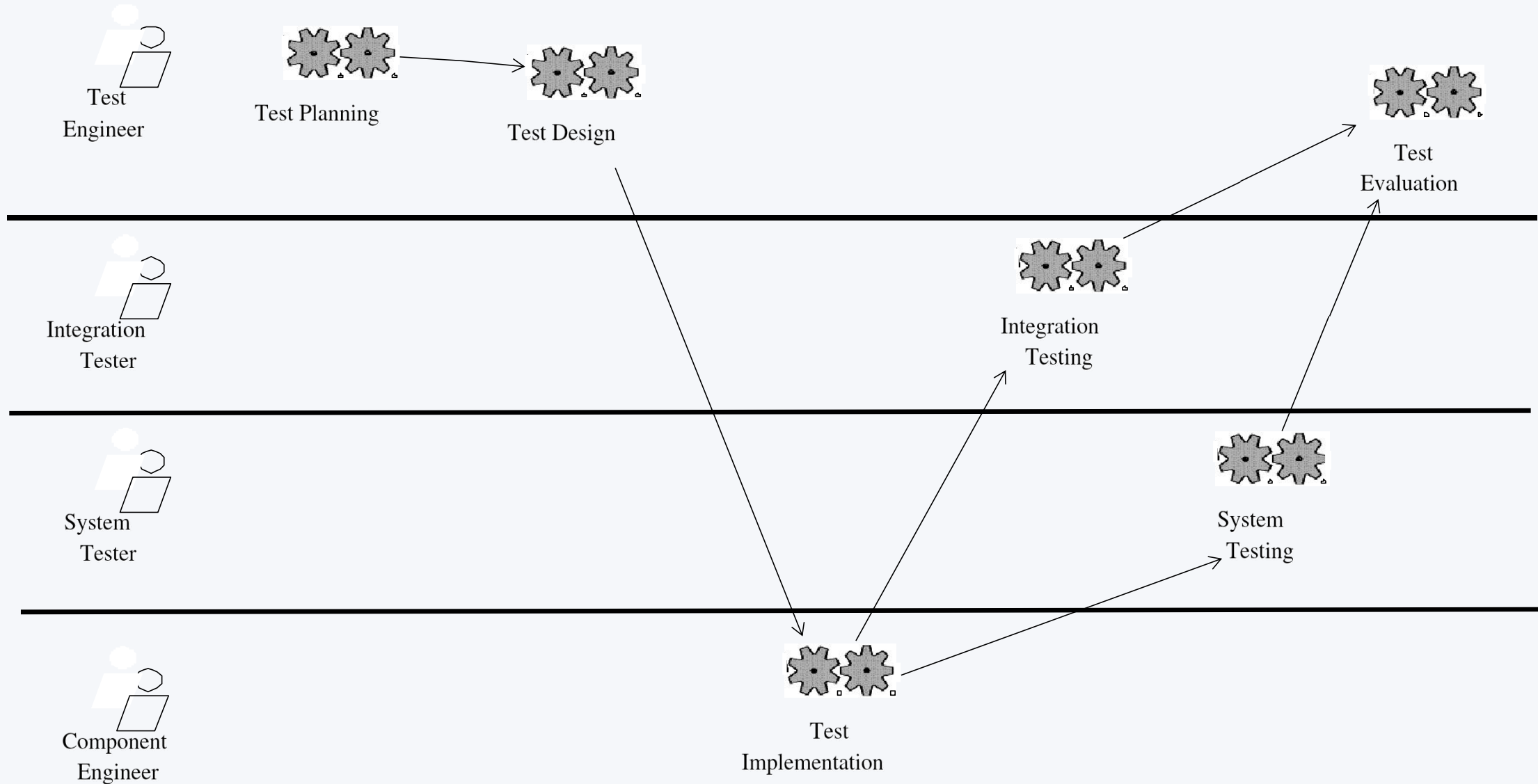
Test Model for ATM System



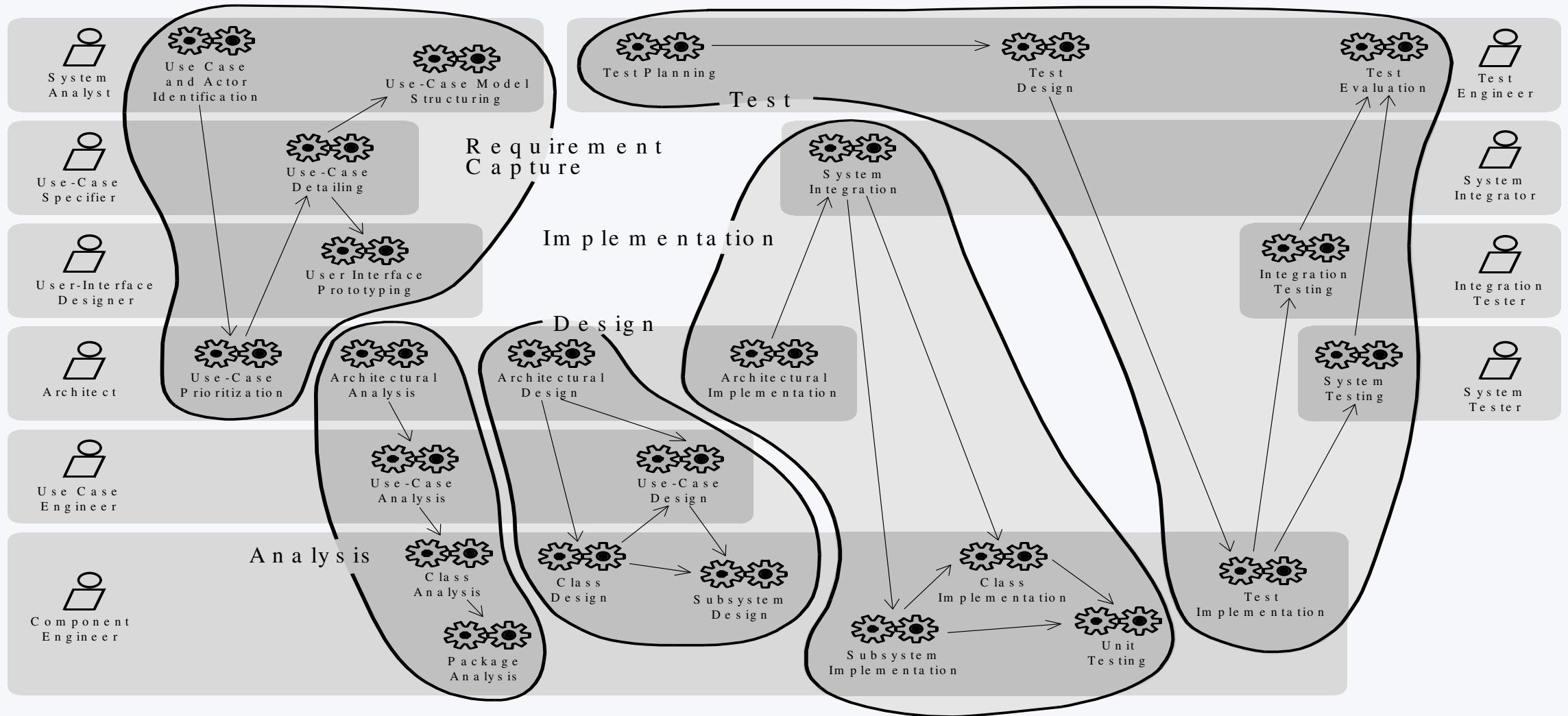
Workers and Responsibilities



Workflow in Test



© 2013 Blackwell Publishing Ltd *Journal of Internal Medicine* 274: 105–114



Summary

Incremental and Evolutionary Models

Incremental and RAD Models

Incremental Approach

Incremental software process models produce software as a series of incremental releases. Each increment builds upon previous ones, delivering additional functionality.

RAD Model

The RAD model is designed for larger projects that must be delivered in tight time. Uses parallel development teams and component-based construction for rapid delivery.

Evolutionary Models

Iterative Nature

Evolutionary process models recognize the iterative nature of most software engineering projects and are designed to accommodate change throughout the development process.

Prototyping Model

Prototyping model helps when requirements are fuzzy by building partial implementations to explore and clarify user needs.

Spiral Model

Spiral model combines iterative development with systematic risk management, using prototyping as a risk reduction mechanism at any stage.

Summary

Component-Based Model and Unified Process

Component-Based Model

Reuse Emphasis

Component-based model emphasizes component reuse and assembly. Focuses on integrating existing, validated components rather than building from scratch.

Library-Driven Development

Uses extensive component repositories and emphasizes integration over coding. **Leverages tested, proven components** to reduce development time and risk.

Unified Process (UP)

Comprehensive Framework

UP is a "use case-driven, architecture-centric, iterative and incremental" software process designed as a framework for UML methods and tools.

Five Phases

UP is an incremental model in which **five phases are defined: inception, elaboration, construction, transition, production.** Each phase has specific objectives and milestones.

Use-Case Driven

Requirements captured as use cases that drive the entire development process from analysis through testing. **Architecture-centric approach** ensures stable, maintainable systems.

Exercise

Model Selection and Application

Question 1: Linear Sequential Model

The linear sequential model of software development is:

- A. A reasonable approach when requirements are well defined.
- B. A good approach when a working program is required quickly.
- C. The best approach to use for projects with large development teams.
- D. An old fashioned model that cannot be used in a modern context.
- E. Both a and d.

Answer: a

Question 2: Alternative Names

The linear sequential model of software development is also known as the:

- A. Classical life cycle model
- B. Fountain model
- C. Spiral model
- D. Waterfall model
- E. Both a and d.

Answer: e

Exercise

Model Selection and Application

Question 3: Incremental Model

The incremental model of software development is:

- A. A reasonable approach when requirements are well defined.
- B. A good approach when a working core product is required quickly.
- C. The best approach to use for projects with large development teams.
- D. A revolutionary model that is not used for commercial products.

Answer: b

Question 4: RAD Model

The rapid application development model is:

- A. Another name for component-based development.
- B. A useful approach when a customer cannot define requirements clearly.
- C. A high speed adaptation of the linear sequential model.
- D. All of the above.

Answer: c

Exercise

True/False and Application Questions

Question 5: Evolutionary Models

Evolutionary software process models:

- A. Are iterative in nature
- B. Can easily accommodate product requirements changes
- C. Do not generally produce throwaway systems
- D. All of the above

Answer: d

Question 6: Prototyping Model

The prototyping model of software development is:

- A. A reasonable approach when requirements are well defined.
- B. A useful approach when a customer cannot define requirements clearly.
- C. The best approach to use for projects with large development teams.
- D. A risky model that rarely produces a meaningful product.

Answer: b

Exercise

True/False and Application Questions

Question 7: UP Phases

Which of these is not one of the phase names defined by the Unified Process model for software development?

- A. Inception phase
- B. Elaboration phase
- C. Construction phase
- D. Validation phase

Answer: d

Question 8: Requirements in UP

In the Unified Process model requirements are determined iteratively and may span more than one phase of the process.

- A. True
- B. False

Answer: a